

React Core

Virtual DOM là gì? Cơ chế diffing của React hoạt động như thế nào?

Virtual DOM là gì?

Virtual DOM là một bản sao “ảo” của DOM thật (DOM của trình duyệt). Thay vì mỗi lần dữ liệu thay đổi là React tác động trực tiếp lên DOM thật (rất tốn chi phí), React sẽ tạo ra một cây Virtual DOM mới trong bộ nhớ. Sau đó React so sánh cây mới này với cây Virtual DOM cũ để tìm ra những điểm thay đổi, rồi mới cập nhật đúng những phần cần thiết lên DOM thật. Nhờ vậy, việc render trở nên nhanh và tối ưu hơn.

Reconciliation là gì?

Reconciliation là quá trình React so sánh (diff) giữa Virtual DOM cũ và Virtual DOM mới để quyết định cần cập nhật những gì trên giao diện. Khi state hoặc props thay đổi, React sẽ tạo ra một cây Virtual DOM mới. Sau đó React chạy thuật toán so sánh hai cây này để tìm ra sự khác biệt, rồi cập nhật lại DOM thật một cách hiệu quả nhất.

Cơ chế diffing (so sánh) hoạt động như thế nào?

Nếu so sánh hai cây DOM một cách tổng quát, thuật toán có thể tốn đến $O(n^3)$, rất chậm. Nhưng React tối ưu bằng cách dùng một thuật toán có độ phức tạp $O(n)$. React làm được điều này vì dựa trên hai giả định chính:

1. Hai element khác type (ví dụ `<div>` và `<span>`) thì React sẽ xem như khác hoàn toàn và thay mới cả subtree bên dưới.
2. Khi render danh sách, developer cung cấp key để giúp React nhận biết phần tử nào là cũ, phần tử nào là mới.

Nhờ hai giả định này, React không cần so sánh mọi trường hợp phức tạp, mà chỉ cần duyệt cây một lần theo thứ tự từ trên xuống dưới (tree comparison theo từng level).

Tree comparison (so sánh cây)

React sẽ bắt đầu từ node gốc, so sánh từng node cùng vị trí giữa cây cũ và cây mới:

- Nếu cùng type (ví dụ đều là `<div>`), React chỉ cập nhật những thuộc tính thay đổi (className, style, props...).
- Nếu khác type, React xóa node cũ và tạo node mới hoàn toàn.
- Sau đó React tiếp tục so sánh các node con theo cùng cách như vậy.

Key đóng vai trò gì?

Khi render một danh sách (ví dụ dùng map), nếu không có key, React sẽ so sánh các phần tử theo vị trí index. Điều này có thể gây ra việc cập nhật sai hoặc không tối ưu khi thêm/xóa phần tử ở giữa danh sách.

key giúp React nhận diện chính xác từng phần tử trong danh sách. Ví dụ nếu bạn có một list user với id duy nhất, thì dùng `key={user.id}` sẽ giúp React biết user nào vẫn tồn tại, user nào bị xóa hoặc thêm mới. Nhờ đó React chỉ cập nhật đúng phần tử thay đổi thay vì render lại toàn bộ danh sách.

Tóm lại:

Virtual DOM giúp React không phải cập nhật DOM thật liên tục. Reconciliation là quá trình so sánh hai cây Virtual DOM. React dùng thuật toán $O(n)$ dựa trên giả định về type và key để tối ưu việc diffing. key đặc biệt quan trọng khi làm việc với danh sách để đảm bảo việc cập nhật chính xác và hiệu quả.

Tại sao key lại quan trọng trong list rendering? Nếu dùng index làm key thì rủi ro gì?

Khi render một danh sách trong React (ví dụ dùng map để render nhiều item), React cần một cách để biết phần tử nào là phần tử cũ, phần tử nào là mới, phần tử nào bị xóa hoặc thay đổi. key chính là “ID” mà React dùng để nhận diện từng phần tử trong danh sách qua các lần render. Nhờ có key, React có thể cập nhật đúng phần tử bị thay đổi thay vì render lại toàn bộ danh sách.

key quan trọng vì nó giúp quá trình so sánh (reconciliation) diễn ra chính xác. Khi state thay đổi và danh sách được render lại, React sẽ nhìn vào key để ghép phần tử cũ với phần tử mới tương ứng. Nếu key ổn định và duy nhất (ví dụ id từ database), React sẽ biết chính xác item nào vẫn giữ nguyên, item nào bị xóa, item nào được thêm mới.

Nếu dùng index làm key, rủi ro sẽ xuất hiện khi danh sách có thêm, xóa hoặc sắp xếp lại phần tử. Vì index dựa vào vị trí, nên khi bạn chèn một item vào đầu danh sách, tất cả các index phía sau sẽ thay đổi. React sẽ hiểu nhầm rằng các phần tử đó là “item mới”, trong khi thực tế chỉ là vị trí thay đổi. Điều này có thể gây ra lỗi hiển thị, mất state cục bộ (ví dụ input đang nhập bị nhảy nội dung), hoặc re-render không cần thiết.

Tuy nhiên, dùng index làm key có thể chấp nhận được trong trường hợp danh sách cố định, không bao giờ thay đổi thứ tự, không thêm/xóa phần tử. Nhưng trong thực tế, tốt nhất nên dùng một giá trị duy nhất và ổn định như id để làm key, để tránh các bug khó phát hiện sau này.

Controlled vs Uncontrolled component? Khi nào nên dùng mỗi loại?

Trong React, Controlled component là những component mà giá trị của input (ví dụ input text, checkbox, select...) được quản lý hoàn toàn bởi state của React. Nghĩa là mỗi khi người dùng nhập gì đó, bạn lưu giá trị đó vào state, và giá trị hiển thị trên input cũng lấy từ chính state đó. React giữ quyền kiểm soát dữ liệu.

Ví dụ dễ hiểu: bạn có một ô input, khi người dùng gõ chữ, bạn dùng onChange để cập nhật state. Sau đó input luôn hiển thị giá trị từ state. Tức là dữ liệu đi theo vòng: người dùng nhập → cập nhật state → render lại → hiển thị giá trị mới.

Ngược lại, Uncontrolled component là những component mà giá trị của input được quản lý bởi chính DOM, không phải bởi state của React. Bạn không lưu giá trị vào state mỗi khi thay đổi. Thay vào đó, khi cần lấy dữ liệu (ví dụ khi submit form), bạn dùng ref để đọc trực tiếp giá trị từ DOM.

Nói đơn giản:

Controlled = React giữ và kiểm soát dữ liệu.

Uncontrolled = DOM tự giữ dữ liệu, React chỉ đọc khi cần.

Khi nào nên dùng Controlled component?

Bạn nên dùng controlled khi:

- Cần validate dữ liệu ngay khi người dùng nhập.
- Cần bật/tắt button dựa vào dữ liệu nhập.
- Cần format dữ liệu (ví dụ tự động viết hoa).
- Cần đồng bộ dữ liệu input với các phần khác của UI.

Controlled phù hợp với form phức tạp và là cách được khuyến khích dùng nhiều trong React vì dễ kiểm soát logic.

Khi nào nên dùng Uncontrolled component?

Bạn có thể dùng uncontrolled khi:

- Form đơn giản, không cần xử lý nhiều logic.
- Chỉ cần lấy dữ liệu một lần khi submit.
- Muốn code gọn nhẹ hơn, ít quản lý state hơn.

Uncontrolled thường đơn giản hơn nhưng kém linh hoạt hơn.

Tóm lại, nếu bạn cần kiểm soát chặt chẽ dữ liệu và xử lý logic liên quan đến input, hãy dùng controlled component. Nếu chỉ cần lấy giá trị đơn giản và không cần xử lý phức tạp, uncontrolled có thể là lựa chọn nhanh gọn hơn.

Synthetic Event trong React là gì? Nó khác gì native DOM event?

Synthetic Event trong React là gì?

Synthetic Event là một “bản bọc” (wrapper) mà React tạo ra xung quanh native DOM event của trình duyệt. Khi bạn viết onClick, onChange, onSubmit trong React, thực tế bạn không làm việc trực tiếp với event gốc của trình duyệt, mà là với một đối tượng Synthetic Event do React cung cấp.

React tạo ra Synthetic Event để đảm bảo event hoạt động giống nhau trên mọi trình duyệt. Vì mỗi trình duyệt có thể xử lý event hơi khác nhau, React chuẩn hóa (normalize) lại để bạn không phải lo sự khác biệt đó.

Nó khác gì với native DOM event?

Thứ nhất, về mục đích:

Native DOM event là event thật do trình duyệt cung cấp (ví dụ khi bạn click chuột).

Synthetic Event là một lớp trung gian do React tạo ra để bọc và chuẩn hóa event đó.

Thứ hai, về tính nhất quán:

Synthetic Event giúp bạn viết code giống nhau trên mọi trình duyệt mà không cần xử lý các khác biệt nhỏ giữa Chrome, Firefox, Safari... Native event thì có thể có sự khác nhau tùy môi trường.

Thứ ba, về cơ chế hoạt động:

React dùng cơ chế gọi là event delegation. Thay vì gắn nhiều event listener vào từng phần tử, React thường gắn một listener ở mức cao (ví dụ root) rồi tự quản lý việc phân phối event xuống component phù hợp. Điều này giúp tối ưu hiệu năng.

Tóm lại, Synthetic Event là một lớp trung gian mà React tạo ra để chuẩn hóa và tối ưu việc xử lý sự kiện. Bạn vẫn có thể truy cập native event thông qua event.nativeEvent, nhưng trong hầu hết trường hợp, dùng Synthetic Event là đủ và được React khuyến khích.

React lifecycle (Class component) vs Hooks lifecycle mapping?

Trong React, Class component có các lifecycle method (vòng đời) rõ ràng như componentDidMount, componentDidUpdate, componentWillUnmount... Những hàm này cho phép bạn chạy logic ở từng thời điểm khác nhau: khi component được tạo ra, khi nó cập nhật, và khi nó bị xóa khỏi màn hình.

Với Function component, React không còn dùng lifecycle method nữa mà thay bằng Hooks, đặc biệt là useEffect. Thay vì chia logic ra nhiều hàm lifecycle như class, bạn dùng useEffect để xử lý hầu hết các tình huống liên quan đến side effect (gọi API, lắng nghe sự kiện, setTimeout, cleanup...).

Trong Class component, vòng đời cơ bản thường là:

- Khi component được mount (hiển thị lần đầu): componentDidMount
- Khi component được update (state/props thay đổi): componentDidUpdate
- Khi component bị unmount (bị xóa khỏi UI): componentWillUnmount

Bạn phải viết logic tương ứng trong từng method riêng biệt.

Còn trong Function component với Hooks, cách mapping sẽ như sau:

- `componentDidMount` → dùng `useEffect` với dependency array rỗng []. Khi đó effect chỉ chạy một lần sau lần render đầu tiên.
 - `componentDidUpdate` → dùng `useEffect` với dependency array chứa biến bạn muốn theo dõi. Mỗi khi biến đó thay đổi, effect sẽ chạy lại.
 - `componentWillUnmount` → return một hàm cleanup bên trong `useEffect`. Hàm này sẽ được gọi khi component bị unmount hoặc trước khi effect chạy lại.
- Ví dụ dễ hiểu: nếu bạn muốn gọi API khi component mount và hủy listener khi unmount, trong class bạn sẽ tách ra 2 lifecycle method khác nhau. Nhưng với Hooks, bạn đặt logic trong `useEffect`, và return một hàm cleanup ở cuối.

Điểm khác biệt lớn là:

Class component chia lifecycle thành nhiều method riêng. Function component với Hooks gom logic liên quan vào cùng một chỗ bằng `useEffect`. Điều này giúp code dễ đọc hơn vì logic liên quan được đặt gần nhau thay vì rải rác ở nhiều lifecycle method.

Tóm lại, lifecycle của class và Hooks có thể làm những việc tương tự nhau, nhưng Hooks (đặc biệt là `useEffect`) linh hoạt và gọn gàng hơn, nên hiện nay đa số dự án React hiện đại ưu tiên dùng Function component với Hooks.

useEffect hoạt động như thế nào? Dependency array ảnh hưởng ra sao?

`useEffect` là một Hook trong React dùng để xử lý side effect. Side effect là những việc không đơn thuần chỉ render giao diện, ví dụ như: gọi API, `setTimeout`, đăng ký event listener, thao tác với DOM, lưu dữ liệu vào `localStorage`... Nói đơn giản, `useEffect` cho phép bạn chạy một đoạn logic sau khi component render xong.

Cách hoạt động cơ bản là: mỗi lần component render, React sẽ kiểm tra `useEffect`. Sau khi render xong và cập nhật giao diện, React mới chạy hàm bên trong `useEffect`. Điều này giúp đảm bảo việc render UI không bị chặn bởi các tác vụ khác như gọi API.

Phần quan trọng nhất của `useEffect` là dependency array (mảng phụ thuộc) – chính là tham số thứ hai.

Nếu bạn truyền vào một mảng rỗng [], effect sẽ chỉ chạy đúng một lần sau lần render đầu tiên. Trường hợp này giống với `componentDidMount` trong class component. Thường dùng khi bạn muốn gọi API lần đầu hoặc khởi tạo dữ liệu.

Nếu bạn truyền vào một mảng có biến, ví dụ `[userId]`, thì effect sẽ chạy lại mỗi khi `userId` thay đổi. React sẽ so sánh giá trị cũ và mới của `userId`, nếu khác thì effect được chạy lại. Trường hợp này giống với `componentDidUpdate` nhưng chỉ theo dõi những biến cụ thể. Nếu bạn không truyền dependency array, `useEffect` sẽ chạy sau mỗi lần render. Điều này có thể gây gọi API liên tục hoặc chạy logic nhiều lần không cần thiết, nên thường không được khuyến khích nếu không có lý do rõ ràng.

Một điểm quan trọng khác là `useEffect` có thể trả về một hàm `cleanup`. Hàm này sẽ được gọi khi component bị `unmount`, hoặc trước khi effect chạy lại lần tiếp theo. Ví dụ, nếu bạn đăng ký event listener, bạn nên gỡ listener trong `cleanup` để tránh memory leak.

Tóm lại, `useEffect` dùng để xử lý các logic chạy sau khi render. Dependency array quyết định khi nào effect được chạy lại. Hiểu đúng dependency array là chìa khóa để tránh bug như gọi API vô hạn hoặc dữ liệu không được cập nhật đúng lúc.

Cleanup function trong `useEffect` dùng để làm gì? Ví dụ thực tế?

Cleanup function trong `useEffect` là một hàm mà bạn return ra bên trong `useEffect`, và nó sẽ được React gọi khi component bị `unmount` (bị gỡ khỏi màn hình) hoặc trước khi `useEffect` chạy lại lần tiếp theo. Mục đích chính của `cleanup` là dọn dẹp tài nguyên để tránh lỗi và memory leak.

Nói đơn giản, nếu trong `useEffect` bạn “mở” một thứ gì đó (ví dụ: đăng ký sự kiện, tạo timer, mở kết nối...), thì trong `cleanup` bạn nên “đóng” hoặc “hủy” nó đi.

Ví dụ thực tế 1: Event Listener.

Giả sử bạn thêm một event listener để lắng nghe sự kiện `resize` của window. Nếu bạn không gỡ nó khi component bị `unmount`, listener đó vẫn tồn tại và có thể gây lỗi hoặc làm app chậm dần theo thời gian. Khi đó, `cleanup` function sẽ dùng để `remove` event listener.

Luồng sẽ là:

- Component mount → thêm event listener
- Component unmount → `cleanup` chạy → `remove` event listener

Ví dụ thực tế 2: `setInterval` hoặc `setTimeout`.

Nếu bạn dùng `setInterval` để chạy một hành động lặp lại (ví dụ cập nhật thời gian mỗi giây), bạn cần dùng `clearInterval` trong `cleanup`. Nếu không, interval vẫn tiếp tục chạy ngay cả khi component đã bị xóa, gây rò rỉ bộ nhớ hoặc lỗi logic.

Ví dụ thực tế 3: Gọi API có hủy request.

Trong một số trường hợp, nếu component `unmount` trước khi API trả kết quả, bạn nên hủy request hoặc kiểm tra trạng thái để tránh việc cập nhật state trên một component đã `unmount` (có thể gây warning trong React).

Tóm lại, `cleanup` function dùng để “dọn dẹp hậu quả” của những gì bạn đã tạo ra trong `useEffect`. Nếu bạn mở kết nối, thêm listener, tạo timer, hoặc đăng ký bất cứ thứ gì có vòng đời dài hơn một lần render, bạn gần như luôn nên có `cleanup` để đảm bảo ứng dụng chạy ổn định và không bị memory leak.

useMemo vs useCallback khác nhau thế nào? Khi nào KHÔNG nên dùng?

useMemo và useCallback đều là Hook dùng để tối ưu hiệu năng, bằng cách ghi nhớ (memoize) một giá trị giữa các lần render. Tuy nhiên, hai cái này ghi nhớ hai thứ khác nhau.

useMemo dùng để ghi nhớ kết quả của một phép tính. Nghĩa là nếu bạn có một đoạn code tính toán nặng (ví dụ lọc một danh sách lớn, tính toán phức tạp), bạn có thể dùng useMemo để chỉ tính lại khi dependency thay đổi. Nếu dependency không đổi, React sẽ trả về kết quả cũ thay vì tính lại.

Còn useCallback dùng để ghi nhớ một function. Mỗi lần component render, JavaScript sẽ tạo ra function mới. Điều này có thể gây re-render không cần thiết khi bạn truyền function đó xuống component con. useCallback giúp giữ nguyên reference của function nếu dependency không thay đổi.

Hiểu đơn giản:

- useMemo → ghi nhớ giá trị.
- useCallback → ghi nhớ hàm.

Thực tế, useCallback(fn, deps) có thể xem như tương đương với useMemo(() => fn, deps).

Khi nào nên dùng?

Bạn nên dùng useMemo khi có phép tính tốn tài nguyên và bạn muốn tránh tính lại không cần thiết. Ví dụ: filter một mảng 10.000 phần tử mỗi lần render.

Bạn nên dùng useCallback khi:

- Truyền function xuống component con có dùng React.memo.
- Hoặc function đó nằm trong dependency của useEffect và bạn không muốn nó thay đổi liên tục.

Khi nào KHÔNG nên dùng?

Không nên lạm dụng useMemo và useCallback cho mọi thứ. Bản thân việc memo hóa cũng có chi phí (React phải lưu và so sánh dependency). Nếu phép tính rất nhẹ hoặc component không bị re-render nhiều, việc dùng chúng có thể làm code phức tạp hơn mà không cải thiện hiệu năng đáng kể.

Ngoài ra, nếu bạn không truyền function xuống component con hoặc không có vấn đề re-render, thì không cần dùng useCallback.

Tóm lại, useMemo và useCallback là công cụ tối ưu, không phải công cụ bắt buộc. Chỉ nên dùng khi bạn thực sự gặp vấn đề hiệu năng hoặc có lý do rõ ràng. Nếu app vẫn chạy mượt, code đơn giản thường tốt hơn code tối ưu quá sớm.

React.memo hoạt động ra sao? So sánh với useMemo?

React.memo là một kỹ thuật tối ưu giúp ngăn component con re-render không cần thiết. Bình thường, khi component cha render lại, tất cả component con bên trong nó cũng sẽ render lại, dù props truyền xuống có thay đổi hay không. React.memo giúp React kiểm tra lại props của component con, nếu props không thay đổi, React sẽ bỏ qua việc render lại component đó.

Nói đơn giản, React.memo giống như một lớp “so sánh props” trước khi render. Nếu props cũ và props mới giống nhau (so sánh nông – shallow comparison), component sẽ không bị render lại.

Ví dụ dễ hiểu:

Bạn có một component cha có state thay đổi liên tục (ví dụ counter). Bên trong có một component con chỉ nhận một prop cố định. Nếu không dùng React.memo, mỗi lần cha render lại thì con cũng render lại. Nhưng nếu bọc component con bằng React.memo, React sẽ thấy props không đổi và bỏ qua render của con, giúp tối ưu hiệu năng.

So sánh với useMemo:

React.memo dùng để ghi nhớ cả một component (ngăn nó re-render nếu props không đổi). useMemo dùng để ghi nhớ một giá trị tính toán bên trong component.

Nói cách khác:

- React.memo → tối ưu ở cấp độ component.
- useMemo → tối ưu ở cấp độ giá trị tính toán.

Một điểm quan trọng là: nếu bạn dùng React.memo nhưng lại truyền xuống một function mới mỗi lần render (không dùng useCallback), thì props vẫn bị xem là thay đổi và component vẫn re-render. Vì vậy, React.memo thường được dùng chung với useCallback để tối ưu tốt hơn.

Tóm lại, React.memo giúp tránh re-render component khi props không thay đổi, còn useMemo giúp tránh tính toán lại giá trị khi dependency không đổi. Hai cái phục vụ mục đích khác nhau nhưng đều nhằm cải thiện hiệu năng khi cần thiết.

Lifting state up là gì? Khi nào nên dùng Context thay vì props drilling?

Lifting state up là khái niệm đưa state lên component cha gần nhất để nhiều component con có thể dùng chung dữ liệu đó. Thay vì mỗi component tự giữ state riêng, ta “nhấc” state lên một cấp cao hơn, rồi truyền xuống qua props.

Ví dụ dễ hiểu: bạn có hai input ở hai component khác nhau nhưng cần đồng bộ dữ liệu với nhau. Nếu mỗi component tự giữ state riêng thì chúng không biết gì về nhau. Lúc này, bạn đưa state lên component cha, rồi truyền giá trị và hàm cập nhật xuống cho cả hai

component con. Khi một bên thay đổi, state ở cha cập nhật, và bên còn lại cũng nhận được giá trị mới.

Nói đơn giản: lifting state up giúp chia sẻ dữ liệu giữa các component có quan hệ với nhau.

Tuy nhiên, khi ứng dụng lớn dần, bạn có thể gặp tình trạng gọi là props drilling. Đây là việc phải truyền props qua nhiều tầng component trung gian, dù những component ở giữa không thực sự dùng dữ liệu đó. Chúng chỉ “chuyền tay” props xuống dưới. Điều này làm code dài dòng, khó đọc và khó bảo trì.

Ví dụ: Component A có state, cần truyền xuống Component D, nhưng phải đi qua B và C. B và C không dùng dữ liệu đó nhưng vẫn phải nhận và truyền tiếp. Đây chính là props drilling.

Khi nào nên dùng Context thay vì props drilling?

Bạn nên dùng Context khi:

- Dữ liệu cần dùng ở rất nhiều nơi trong cây component.
- Dữ liệu mang tính toàn cục như: user đăng nhập, theme (dark/light), ngôn ngữ, cấu hình app.
- Bạn muốn tránh truyền props qua nhiều tầng trung gian.

Context cho phép bạn “cung cấp” dữ liệu ở một cấp cao và “truy cập trực tiếp” ở bất kỳ component con nào bên dưới mà không cần truyền qua từng cấp props.

Tóm lại:

- Lifting state up dùng khi vài component gần nhau cần chia sẻ state chung.
- Props drilling xảy ra khi phải truyền props qua quá nhiều tầng.
- Context nên dùng khi dữ liệu mang tính global hoặc được sử dụng rộng rãi trong nhiều component, giúp code gọn gàng và dễ quản lý hơn.

Sự khác nhau giữa useRef và useState là gì? Khi nào nên dùng useRef thay vì useState để tránh re-render?

useState và useRef đều dùng để lưu trữ giá trị trong function component, nhưng cách hoạt động của chúng khác nhau ở điểm rất quan trọng: useState sẽ gây re-render khi giá trị thay đổi, còn useRef thì không.

Khi bạn dùng useState, mỗi lần gọi setState với giá trị mới, React sẽ render lại component để cập nhật UI. Điều này phù hợp khi giá trị đó ảnh hưởng trực tiếp đến giao diện, ví dụ như số đếm, trạng thái mở/đóng modal, dữ liệu hiển thị lên màn hình. useState sinh ra để quản lý state phục vụ cho việc render.

Ngược lại, `useRef` lưu một giá trị trong thuộc tính `current` và thay đổi nó sẽ không làm component re-render. React chỉ tạo object ref một lần và giữ nguyên reference đó qua các lần render. Vì vậy `useRef` phù hợp cho những dữ liệu không cần hiển thị lên UI, ví dụ như lưu DOM element, lưu timer ID, lưu giá trị trước đó, hoặc lưu một biến tạm để tránh tạo lại nhiều lần.

Bạn nên dùng `useRef` thay vì `useState` khi giá trị đó chỉ dùng nội bộ cho logic và không ảnh hưởng trực tiếp đến giao diện. Ví dụ, khi cần lưu một biến đếm số lần gọi API để kiểm soát logic, hoặc lưu một flag để biết component đã mount hay chưa. Nếu dùng `useState` trong những trường hợp này, mỗi lần cập nhật sẽ gây re-render không cần thiết.

Tóm lại, nếu thay đổi giá trị mà bạn muốn UI cập nhật theo, dùng `useState`. Nếu chỉ cần lưu dữ liệu giữa các lần render mà không muốn gây re-render, dùng `useRef`. Đây là cách giúp tránh những lần render không cần thiết và tối ưu hiệu năng trong một số trường hợp.

Batch update trong React là gì? React xử lý batching state update khác nhau thế nào giữa event handler và async code?

Batch update trong React là cơ chế gom nhiều lần cập nhật state lại với nhau và chỉ render một lần duy nhất. Mục đích là để tối ưu hiệu năng. Nếu mỗi lần gọi `setState` mà React render ngay lập tức thì ứng dụng sẽ rất chậm khi có nhiều cập nhật liên tiếp.

Ví dụ, nếu trong một hàm bạn gọi `setState` hai hoặc ba lần liên tiếp, React sẽ không render ba lần. Thay vào đó, nó sẽ gom các thay đổi lại và render một lần sau khi hàm đó chạy xong. Đây chính là batching.

Trước React 18, batching chỉ hoạt động tự động bên trong event handler của React, ví dụ như `onClick` hoặc `onChange`. Nếu bạn gọi `setState` trong một đoạn async code như `setTimeout`, `promise` hoặc sau khi gọi API, thì mỗi lần `setState` có thể gây ra một lần render riêng biệt. Điều này làm cho async code có thể render nhiều lần hơn mong muốn.

Tuy nhiên, từ React 18 trở đi, React đã hỗ trợ automatic batching cho cả async code. Nghĩa là dù bạn gọi `setState` trong `promise`, `setTimeout` hay `async/await`, React vẫn sẽ gom các cập nhật lại và render một lần duy nhất (trong cùng một tick xử lý).

Tóm lại, batching là cơ chế giúp React gom nhiều state update thành một lần render để tối ưu hiệu năng. Trước đây chỉ áp dụng trong event handler của React, còn từ React 18 thì batching được mở rộng cho cả async code, giúp ứng dụng hoạt động hiệu quả hơn mà developer không cần xử lý thủ công.

React Advanced

React concurrent rendering là gì? (React 18)

React Concurrent Rendering (từ React 18) là một cơ chế mới giúp React render thông minh và mượt hơn bằng cách không chặn toàn bộ UI khi đang xử lý cập nhật. Trước đây, khi React bắt đầu render một update, nó sẽ làm một mạch đến khi xong (blocking). Nếu component lớn hoặc tính toán nhiều, giao diện có thể bị “đơ” trong một khoảng thời gian ngắn.

Với Concurrent Rendering, React có khả năng tạm dừng, tiếp tục, hoặc hủy bỏ một quá trình render nếu cần. Điều này giúp React ưu tiên những cập nhật quan trọng hơn, ví dụ như thao tác người dùng (gõ phím, click), thay vì xử lý tất cả update cùng mức độ ưu tiên.

Hiểu đơn giản thế này:

Trước đây React giống như một người làm việc theo kiểu “làm cho xong việc A rồi mới làm việc B”. Còn với concurrent rendering, React giống như người có thể tạm dừng việc A nếu thấy việc B quan trọng hơn, xử lý B trước rồi quay lại A sau. Nhờ đó trải nghiệm người dùng mượt hơn.

Một điểm quan trọng là: Concurrent Rendering không phải là một API cụ thể mà bạn bật lên bằng một dòng code đơn giản. Nó là cơ chế nền tảng hoạt động khi bạn dùng React 18 và tạo app bằng createRoot. Ngoài ra, React 18 cung cấp một số tính năng hỗ trợ concurrent như:

- startTransition: đánh dấu một update là không gấp (low priority).
- useTransition: giúp quản lý trạng thái pending của transition.
- useDeferredValue: trì hoãn cập nhật một giá trị để tránh render nặng ngay lập tức.

Ví dụ thực tế:

Giả sử bạn có một ô search. Khi người dùng gõ chữ, bạn vừa cập nhật text trong input (cần phản hồi ngay), vừa lọc một danh sách 10.000 phần tử (nặng). Với concurrent rendering, React có thể ưu tiên cập nhật input trước để không bị lag, còn việc lọc danh sách có thể xử lý sau.

Tóm lại, Concurrent Rendering trong React 18 giúp React quản lý render theo mức độ ưu tiên, có thể tạm dừng và tiếp tục công việc, từ đó giúp ứng dụng mượt hơn và phản hồi tốt hơn khi có nhiều cập nhật xảy ra cùng lúc.

StrictMode trong React dùng để làm gì? Tại sao useEffect chạy 2 lần ở dev?

StrictMode là một công cụ hỗ trợ trong React dùng để giúp developer phát hiện vấn đề tiềm ẩn trong quá trình phát triển (development). Nó không ảnh hưởng đến giao diện người dùng và cũng không chạy trong môi trường production. Mục tiêu chính của StrictMode là

cảnh báo những đoạn code có thể gây bug trong tương lai hoặc không tương thích với các tính năng mới như Concurrent Rendering.

Khi bạn bọc ứng dụng bằng `<StrictMode>`, React sẽ bật thêm một số kiểm tra đặc biệt. Ví dụ: phát hiện việc dùng lifecycle cũ, phát hiện side effect không an toàn, hoặc kiểm tra logic cleanup có đúng hay không.

Vậy tại sao `useEffect` chạy 2 lần trong môi trường dev?

Trong React 18, khi bật `StrictMode`, React sẽ cố tình mount → unmount → mount lại component ngay lập tức trong môi trường development. Việc này làm cho `useEffect` chạy hai lần (nếu bạn dùng dependency []).

Mục đích không phải là lỗi, mà là để kiểm tra xem effect của bạn có xử lý cleanup đúng cách hay không.

Luồng thực tế khi dev với `StrictMode` sẽ là:

- Component mount → `useEffect` chạy
- Component unmount ngay lập tức → cleanup chạy
- Component mount lại → `useEffect` chạy lần nữa

Nếu bạn không viết cleanup đúng cách (ví dụ quên remove event listener, quên `clearInterval`), bạn sẽ dễ thấy bug xuất hiện trong dev.

Điều quan trọng là: việc chạy 2 lần này chỉ xảy ra trong development, không xảy ra trong production. Khi build production, `useEffect` sẽ chạy đúng một lần như bạn mong đợi.

Tóm lại, `StrictMode` là công cụ giúp phát hiện bug sớm và đảm bảo code an toàn hơn cho tương lai. Việc `useEffect` chạy 2 lần trong dev là hành vi cố ý của React để kiểm tra side effect và cleanup, không phải lỗi của React.

Suspense hoạt động như thế nào? Có thể dùng cho data fetching không?

Suspense trong React là một cơ chế giúp React “chờ” một thứ gì đó chưa sẵn sàng trước khi render UI, và trong lúc chờ thì hiển thị một giao diện dự phòng (fallback), ví dụ như spinner hoặc loading.

Hiểu đơn giản, Suspense giống như bạn nói với React: “Nếu phần này chưa load xong thì hiển thị loading giúp tôi.” Khi dữ liệu hoặc component đã sẵn sàng, React sẽ tự động thay loading bằng nội dung thật.

Cách hoạt động cơ bản là:

Bạn bọc một component bên trong `<Suspense fallback={...}>`. Nếu bên trong có thứ gì đó chưa sẵn sàng (ví dụ component đang được load động bằng `React.lazy`), React sẽ hiển thị fallback cho đến khi việc load hoàn tất.

Ví dụ phổ biến nhất là code splitting với React.lazy. Khi bạn load một component lớn từ file khác, React sẽ tải nó bất đồng bộ. Trong thời gian tải, Suspense sẽ hiển thị loading thay vì để màn hình trắng.

Vậy Suspense có dùng cho data fetching được không?

Câu trả lời là: Có, nhưng phụ thuộc cách bạn fetch data.

Trước đây (React cũ), Suspense chủ yếu dùng cho lazy loading component, còn data fetching thì bạn phải tự quản lý state loading bằng useEffect.

Từ React 18 trở đi, Suspense có thể dùng cho data fetching nếu bạn dùng các thư viện hỗ trợ cơ chế “suspend” như:

- React Server Components
- Một số data library như React Query (ở chế độ suspense)
- Hoặc các framework như Next.js (App Router)

Cơ chế lúc này là: nếu dữ liệu chưa sẵn sàng, component sẽ “throw” một Promise. React bắt Promise đó và hiển thị fallback cho đến khi Promise resolve.

Tuy nhiên, nếu bạn chỉ dùng fetch trong useEffect theo cách truyền thống, Suspense sẽ không tự động hoạt động cho data fetching. Bạn phải dùng cách tiếp cận tương thích với Suspense hoặc dùng framework hỗ trợ.

Tóm lại, Suspense giúp hiển thị UI loading một cách tự nhiên khi component hoặc dữ liệu chưa sẵn sàng. Nó hoạt động rất tốt với lazy loading và hiện nay cũng có thể dùng cho data fetching, nhưng cần đúng cơ chế hoặc thư viện hỗ trợ.

Code splitting trong React làm như thế nào?

Code splitting trong React là kỹ thuật chia nhỏ ứng dụng thành nhiều “mảnh” (bundle nhỏ) thay vì tải toàn bộ code ngay từ đầu. Mục tiêu là khi người dùng mở web, họ chỉ tải những gì cần thiết cho màn hình hiện tại, còn những phần khác sẽ được tải sau khi cần. Điều này giúp web load nhanh hơn, đặc biệt với các ứng dụng lớn.

Trong React, cách phổ biến nhất để làm code splitting là dùng React.lazy() kết hợp với Suspense. Thay vì import component theo cách bình thường (import ngay khi app khởi động), ta sẽ import theo kiểu dynamic import. Ví dụ, thay vì import AdminPage from “./AdminPage”, ta viết const AdminPage = React.lazy(() => import(“./AdminPage")). Khi đó, file AdminPage chỉ được tải khi component này thực sự được render.

Vì component được load bất đồng bộ (async), nên ta cần bọc nó trong Suspense để hiển thị giao diện chờ (loading). Ví dụ: <Suspense fallback={<div>Loading...</div>}>

`<AdminPage /> </Suspense>`. Khi người dùng truy cập trang đó, React sẽ tải file tương ứng, trong lúc chờ sẽ hiển thị “Loading...”. Điều này giúp trải nghiệm mượt hơn.

Trong thực tế, code splitting thường được dùng nhiều nhất khi kết hợp với React Router. Mỗi route (mỗi trang như /login, /dashboard, /admin...) sẽ được lazy load riêng. Như vậy, người dùng vào trang login sẽ không phải tải toàn bộ code của dashboard hay admin ngay từ đầu.

Tóm lại, code splitting giúp tối ưu performance bằng cách tải code theo nhu cầu. Trong React, cách làm đơn giản và phổ biến nhất là dùng `React.lazy()` và `Suspense` để lazy load component, đặc biệt là ở level route.

Lazy loading component hoạt động ra sao?

Lazy loading component trong React là cơ chế chỉ tải component khi thật sự cần dùng đến, thay vì tải tất cả component ngay từ đầu khi ứng dụng khởi động. Điều này giúp giảm dung lượng file JavaScript ban đầu và làm cho trang web load nhanh hơn.

Bình thường khi chúng ta import một component theo cách thông thường, toàn bộ code của component đó sẽ được đưa vào bundle chính và tải ngay khi người dùng mở web. Nhưng với lazy loading, React sẽ trì hoãn việc tải file đó cho đến khi component được render (ví dụ khi người dùng chuyển sang một trang khác).

Cơ chế hoạt động của lazy loading trong React thường dùng `React.lazy()` kết hợp với `Suspense`. Khi ta dùng `React.lazy(() => import("./MyComponent"))`, React sẽ tạo một component đặc biệt. Lúc đầu, component này chưa có code thật sự. Khi React render tới nó, React sẽ gọi hàm `import()` để tải file tương ứng một cách bất đồng bộ (async).

Vì việc tải này mất thời gian (dù chỉ vài mili giây), nên React yêu cầu chúng ta bọc component đó trong `Suspense`. `Suspense` cho phép hiển thị một giao diện tạm thời (fallback), ví dụ như “Loading...”, trong lúc component đang được tải. Khi file được tải xong, React sẽ render component bình thường.

Trong thực tế, lazy loading thường được dùng cho các trang lớn như dashboard, admin, report hoặc những component nặng (có nhiều thư viện bên trong). Nhờ vậy, người dùng không phải tải những phần họ chưa sử dụng, giúp cải thiện hiệu năng và trải nghiệm.

Error Boundary là gì? Tại sao Hook không bắt được error như Class component?

Error Boundary là một component đặc biệt trong React dùng để “bắt” lỗi JavaScript xảy ra trong quá trình render của các component con bên dưới nó. Khi một component bị lỗi (ví

dụ lỗi trong render, lifecycle, hoặc constructor), thay vì làm crash toàn bộ ứng dụng, Error Boundary sẽ hiển thị một giao diện dự phòng (fallback UI) như “Có lỗi xảy ra”, giúp app không bị trắng màn hình.

Error Boundary chỉ có thể được tạo bằng Class Component, bằng cách định nghĩa các lifecycle như `componentDidCatch` hoặc `getDerivedStateFromError`. Khi React phát hiện lỗi trong cây component con, nó sẽ gọi các method này để cho phép chúng ta xử lý lỗi và hiển thị giao diện thay thế. Ví dụ, ta có thể log lỗi về server và hiển thị một màn hình thông báo thân thiện cho người dùng.

Hook không bắt được error như Class Component vì React không cung cấp một hook nào tương đương với `componentDidCatch`. Hooks như `useEffect`, `useState`... chỉ xử lý logic và state, chứ không có cơ chế để “bao” và bắt lỗi của toàn bộ cây component con. Error Boundary hoạt động ở mức lifecycle đặc biệt của Class Component, còn Hook chỉ chạy bên trong function component nên không có quyền can thiệp vào cơ chế bắt lỗi của React.

Nói đơn giản, Error Boundary là cơ chế cấp hệ thống của React để bảo vệ UI khỏi crash. Nó được thiết kế dựa trên lifecycle của Class Component, nên hiện tại chỉ Class Component mới có thể làm Error Boundary. Nếu dùng function component, ta vẫn phải tạo một Class Component để làm Error Boundary rồi bọc các function component bên trong nó.

Portals là gì? Dùng trong trường hợp nào?

Portals trong React là cách cho phép chúng ta render một component ra ngoài cây DOM hiện tại, nhưng vẫn giữ được toàn bộ logic và state của React như bình thường. Nói đơn giản, component đó vẫn thuộc về cây React, nhưng phần HTML của nó lại được chèn vào một vị trí khác trong DOM.

Bình thường khi ta render một component, nó sẽ nằm bên trong thẻ cha của nó. Nhưng có những trường hợp chúng ta không muốn nó bị giới hạn bởi cấu trúc HTML đó, ví dụ như bị ảnh hưởng bởi `overflow: hidden`, `z-index`, hoặc CSS của component cha. Khi đó, ta dùng Portal để “đẩy” phần giao diện đó ra một thẻ khác, thường là ngay dưới `<body>`.

Portals thường được dùng trong các trường hợp như Modal (popup), Dialog, Tooltip, Dropdown menu, hoặc Notification. Ví dụ, khi mở một modal, ta muốn nó nổi lên trên toàn bộ trang web và không bị che bởi các div khác. Nếu render bình thường, modal có thể bị giới hạn trong layout hiện tại. Nhưng nếu dùng Portal, ta có thể render modal vào một div riêng như `<div id="modal-root">` nằm ngoài app chính.

Điều quan trọng là dù được render ra ngoài DOM chính, Portal vẫn giữ được context, state và event bubbling của React. Tức là event vẫn truyền đúng theo cây React, không phải theo cây DOM thật.

Tóm lại, Portals giúp chúng ta render UI ra vị trí khác trong DOM khi cần xử lý layout đặc biệt, thường dùng cho modal, popup, tooltip hoặc các thành phần nổi (overlay).

Higher Order Component (HOC) vs Render Props vs Hooks?

Higher Order Component (HOC), Render Props và Hooks đều là các cách để tái sử dụng logic giữa nhiều component trong React. Tức là thay vì copy-paste logic giống nhau ở nhiều nơi, ta tách nó ra và dùng lại theo các cách khác nhau.

Higher Order Component (HOC) là một hàm nhận vào một component và trả về một component mới. Component mới này sẽ “bọc” component cũ và thêm logic chung vào đó. Ví dụ, ta có một HOC để kiểm tra đăng nhập, nó sẽ bọc các component cần bảo vệ. Nếu chưa đăng nhập thì redirect, nếu đã đăng nhập thì render component gốc. HOC thường được dùng nhiều trước khi Hooks ra đời. Tuy nhiên, nếu dùng nhiều HOC lồng nhau sẽ dẫn tới tình trạng “wrapper hell”, code khó đọc.

Render Props là một kỹ thuật mà component sẽ nhận vào một hàm (thông qua prop) và hàm đó trả về UI. Component cha sẽ xử lý logic, còn phần giao diện sẽ do hàm này quyết định. Cách này giúp chia sẻ logic linh hoạt hơn HOC vì ta kiểm soát được cách render. Tuy nhiên, nếu lồng nhiều render props vào nhau thì JSX sẽ rất rối.

Hooks là cách hiện đại nhất để tái sử dụng logic trong function component. Ta có thể tạo custom hook, ví dụ useAuth() hoặc useFetch(), rồi dùng nó trong nhiều component khác nhau. Hooks không tạo thêm wrapper component như HOC và không làm JSX rối như Render Props. Logic được tách riêng nhưng nhìn code vẫn gọn gàng và dễ hiểu.

Tóm lại, cả ba đều dùng để tái sử dụng logic. HOC và Render Props là cách cũ hơn, thường dùng trong Class Component. Hooks là cách mới và được khuyến khích hiện nay vì code gọn, dễ đọc và dễ bảo trì hơn.

Cách thiết kế reusable component library trong dự án lớn?

Thiết kế reusable component library trong dự án lớn nghĩa là mình tạo ra một bộ component dùng chung (Button, Input, Modal, Table, Card...) để toàn bộ team sử dụng lại, thay vì mỗi người tự viết một kiểu khác nhau. Mục tiêu là đảm bảo UI đồng nhất, dễ bảo trì và giảm lặp code.

Bước đầu tiên là tách rõ component “thuần UI” và component “business logic”. Component library nên tập trung vào UI và hành vi cơ bản, ví dụ Button chỉ lo hiển thị, trạng thái loading, disabled... chứ không chứa logic gọi API hay xử lý nghiệp vụ. Điều này giúp component dễ tái sử dụng ở nhiều nơi khác nhau.

Thứ hai là thiết kế API của component rõ ràng và linh hoạt. Ví dụ Button nên có các props như variant (primary, secondary), size (small, medium, large), onClick, disabled... Thay vì tạo nhiều loại Button khác nhau, ta dùng props để cấu hình. API càng đơn giản, dễ hiểu thì càng dễ dùng trong toàn team.

Thứ ba là quản lý style thống nhất. Thường trong dự án lớn sẽ có design system (màu sắc, font, spacing). Component library nên tuân theo design system này. Có thể dùng CSS module, styled-components hoặc Tailwind, nhưng cần thống nhất một cách để tránh mỗi người làm một kiểu.

Thứ tư là tổ chức cấu trúc thư mục rõ ràng. Ví dụ mỗi component nằm trong một folder riêng, gồm file component, file style và file test. Nếu dự án rất lớn, có thể tách thành một package riêng (ví dụ trong monorepo), để nhiều project cùng sử dụng.

Cuối cùng là cần có documentation và ví dụ sử dụng. Trong dự án lớn, nếu không có tài liệu hướng dẫn thì rất khó để người khác dùng đúng cách. Có thể dùng Storybook để hiển thị các component và các trạng thái khác nhau của chúng.

Tóm lại, reusable component library trong dự án lớn cần tập trung vào: tách UI khỏi business logic, thiết kế props linh hoạt, tuân thủ design system, cấu trúc rõ ràng và có tài liệu đầy đủ. Như vậy team mới có thể dùng chung hiệu quả và lâu dài.

Cách tổ chức folder structure cho dự án React lớn?

Khi làm một dự án React lớn, việc tổ chức folder structure rất quan trọng vì nếu sắp xếp không hợp lý, code sẽ rất nhanh rối và khó bảo trì. Mục tiêu chính là giúp người mới vào project có thể hiểu cấu trúc nhanh, dễ tìm file và dễ mở rộng khi dự án phát triển thêm.

Cách phổ biến và dễ hiểu nhất là tổ chức theo “feature” thay vì theo “loại file”. Nghĩa là thay vì có một folder components, một folder services, một folder reducers quá lớn, ta chia theo từng chức năng. Ví dụ: auth, user, dashboard, orders... Mỗi folder feature sẽ chứa luôn component, hook, service, test liên quan đến chính nó. Cách này giúp logic của từng chức năng nằm gọn trong một chỗ, dễ quản lý hơn.

Ngoài các folder feature, ta thường có một số folder dùng chung như components (chứa reusable UI component như Button, Input), hooks (custom hooks dùng chung), utils (hàm helper), services hoặc api (gọi API), assets (hình ảnh, icon), và layouts (layout tổng như MainLayout, AuthLayout). Những thứ dùng chung toàn hệ thống thì đặt ở mức global, còn thứ chỉ dùng trong một feature thì nên để trong feature đó.

Với dự án lớn hơn nữa, có thể tách riêng phần state management (ví dụ Redux store) thành folder store, và trong đó chia theo từng slice tương ứng với feature. Nếu dùng monorepo, ta có thể tách thành nhiều package nhỏ như ui-library, shared, app-web... để quản lý rõ ràng hơn.

Quan trọng nhất không phải là chọn một cấu trúc “đúng tuyệt đối”, mà là chọn một cách tổ chức rõ ràng, dễ hiểu và thống nhất trong team. Một cấu trúc tốt là khi thêm một feature mới, ta biết chính xác phải đặt file ở đâu mà không cần suy nghĩ quá nhiều.

useTransition trong React 18 dùng để làm gì? Nó giúp cải thiện UX trong trường hợp nào?

useTransition trong React 18 là một hook giúp bạn đánh dấu một số cập nhật state là “không khẩn cấp” (non-urgent). Điều này cho phép React ưu tiên những cập nhật quan trọng trước, còn những cập nhật nặng hơn có thể xử lý sau, giúp giao diện không bị giật hoặc lag.

Bình thường khi bạn gọi setState, React sẽ render lại component ngay. Nhưng nếu việc render đó rất nặng (ví dụ lọc một danh sách lớn hàng nghìn phần tử), UI có thể bị khựng trong lúc xử lý. useTransition cho phép bạn nói với React rằng: “Phần cập nhật này có thể chờ một chút cũng được.” Khi đó React sẽ ưu tiên xử lý các tương tác như gõ phím, click trước, rồi mới render phần nặng phía sau.

Hook này trả về hai thứ: một hàm startTransition và một biến isPending. Bạn đặt logic cập nhật state nặng vào bên trong startTransition. Trong lúc React đang xử lý phần đó, isPending sẽ là true, và bạn có thể hiển thị một trạng thái loading nhẹ để cải thiện trải nghiệm người dùng.

useTransition đặc biệt hữu ích trong các trường hợp như: tìm kiếm với danh sách lớn, chuyển tab có nhiều dữ liệu, hoặc filter/sort bảng dữ liệu lớn. Khi người dùng nhập input tìm kiếm, bạn muốn chữ họ gõ hiển thị ngay lập tức (cập nhật khẩn cấp), nhưng kết quả lọc danh sách có thể xử lý chậm hơn một chút mà không làm giật UI.

Tóm lại, `useTransition` giúp phân biệt giữa cập nhật quan trọng và cập nhật có thể trì hoãn. Nó cải thiện UX trong các trường hợp render nặng, giúp giao diện phản hồi mượt mà hơn khi người dùng tương tác.

Server Components trong React là gì? Khác gì so với Client Components?

Server Components trong React là loại component được render ở phía server thay vì ở trình duyệt. Nghĩa là code của component đó chạy trên server, tạo ra HTML sẵn rồi gửi xuống cho client. Trình duyệt chỉ nhận kết quả đã render, chứ không cần tải toàn bộ logic JavaScript của component đó.

Ngược lại, Client Components là component truyền thống trong React. Chúng chạy ở phía trình duyệt, có thể dùng state, `useEffect`, xử lý sự kiện như `onClick`, và tương tác trực tiếp với người dùng. Toàn bộ code của Client Component sẽ được gửi xuống browser.

Điểm khác biệt quan trọng nhất là Server Components không gửi toàn bộ JavaScript xuống client, nên giúp giảm bundle size đáng kể. Chúng phù hợp cho những phần chỉ cần hiển thị dữ liệu, ví dụ như danh sách sản phẩm, nội dung bài viết, thông tin từ database. Vì chạy ở server, chúng có thể truy cập trực tiếp database hoặc API nội bộ mà không cần gọi từ client.

Tuy nhiên, Server Components không thể dùng các hook như `useState` hoặc `useEffect`, vì chúng không tồn tại lâu dài trong browser và không xử lý tương tác người dùng. Nếu cần tương tác (ví dụ click, nhập form), ta phải dùng Client Component.

Trong thực tế, ứng dụng có thể kết hợp cả hai. Ví dụ phần hiển thị dữ liệu ban đầu là Server Component để tối ưu performance, còn phần tương tác như form hoặc nút bấm sẽ là Client Component.

Tóm lại, Server Components chạy ở server, giúp giảm bundle và tối ưu load ban đầu. Client Components chạy ở browser, dùng cho phần tương tác và state. Hai loại này có thể kết hợp với nhau để tối ưu cả hiệu năng lẫn trải nghiệm người dùng.

Performance Optimization

Làm sao để detect re-render không cần thiết?

Trong React, re-render là chuyện bình thường vì mỗi khi state hoặc props thay đổi thì component sẽ render lại. Nhưng vấn đề là đôi khi component bị render lại dù dữ liệu không thực sự thay đổi. Những lần re-render “không cần thiết” này có thể làm app chậm, đặc biệt trong dự án lớn.

Cách đơn giản nhất để detect là dùng React DevTools và bật tính năng “Highlight updates when components render”. Khi bật lên, mỗi lần component re-render nó sẽ được highlight trên màn hình. Nếu bạn thấy một component cứ nháy liên tục dù không có lý do rõ ràng, rất có thể nó đang bị re-render không cần thiết.

Một cách khác là dùng React Profiler (cũng trong React DevTools). Profiler cho bạn biết component nào render, render bao lâu, và vì sao render. Bạn có thể record một thao tác (ví dụ click button) rồi xem những component nào bị ảnh hưởng. Nếu một component không liên quan mà vẫn render lại, đó là dấu hiệu cần tối ưu.

Ngoài ra, trong lúc debug, bạn có thể tạm thời đặt `console.log` trong component để xem nó render bao nhiêu lần. Nếu mỗi lần component cha render mà component con cũng render theo dù props không đổi, thì có thể bạn cần dùng `React.memo` hoặc `useMemo`, `useCallback` để tránh tạo ra object hoặc function mới mỗi lần render.

Tóm lại, để detect re-render không cần thiết, bạn nên dùng React DevTools (Highlight update và Profiler) để quan sát thực tế. Sau đó phân tích nguyên nhân thường là do props thay đổi reference (object, array, function) hoặc do state quản lý chưa hợp lý.

Tại sao component bị re-render dù props không thay đổi?

Trong React, một component sẽ re-render mỗi khi component cha của nó re-render, ngay cả khi props truyền xuống nhìn “có vẻ” không thay đổi. Đây là điều nhiều người mới học React hay bị nhầm. Mặc định, React không tự động kiểm tra sâu xem props có giống hệt trước đó hay không, mà chỉ cần component cha chạy lại là component con cũng sẽ chạy lại hàm render.

Một nguyên nhân rất phổ biến là props tuy không đổi về giá trị, nhưng lại đổi về reference. Ví dụ bạn truyền xuống một object, array hoặc function được tạo mới trong mỗi lần render. Dù nội dung giống nhau, nhưng vì mỗi lần là một object mới trong bộ nhớ, React sẽ coi nó là khác và render lại component con. Đây là lý do vì sao nhiều khi phải dùng `useMemo` hoặc `useCallback` để giữ reference ổn định.

Một nguyên nhân khác là khi component không được bọc bằng `React.memo`. Mặc định function component sẽ luôn render lại khi cha render. Nếu bạn muốn React so sánh props cũ và mới (theo kiểu shallow comparison), bạn cần dùng `React.memo`. Nếu props không đổi (theo so sánh nông), component sẽ không render lại.

Ngoài ra, state bên trong component thay đổi cũng sẽ làm nó re-render, dù props không đổi. Hoặc nếu component dùng context, khi giá trị context thay đổi thì tất cả component dùng context đó cũng sẽ render lại.

Tóm lại, component có thể re-render dù props “nhìn giống nhau” vì React không so sánh sâu, vì reference thay đổi, vì component cha render lại, vì state nội bộ thay đổi hoặc vì context thay đổi. Để tối ưu, ta thường dùng React.memo, useMemo và useCallback khi cần thiết.

Khi nào nên memoize? Khi nào over-optimization gây hại?

Memoize trong React nghĩa là “ghi nhớ” lại giá trị hoặc function cũ để tránh tạo lại hoặc tính toán lại không cần thiết. Trong React, ta thường dùng React.memo để ghi nhớ component, useMemo để ghi nhớ giá trị tính toán, và useCallback để ghi nhớ function.

Ta nên memoize khi có vấn đề hiệu năng thực sự. Ví dụ, một component render rất nhiều lần và có tính toán nặng (lọc danh sách lớn, tính toán phức tạp), hoặc khi component con bị re-render liên tục do props là object hoặc function bị tạo mới mỗi lần. Trong những trường hợp đó, memoize giúp giảm số lần render và cải thiện performance rõ ràng.

Tuy nhiên, không phải lúc nào cũng nên dùng memoize. Mỗi lần dùng useMemo hay useCallback, React cũng phải tốn chi phí để lưu trữ và so sánh dependency. Nếu component nhỏ, logic đơn giản, render nhanh, thì việc memoize có thể làm code phức tạp hơn mà không mang lại lợi ích đáng kể. Thậm chí trong một số trường hợp, chi phí so sánh dependency còn tốn hơn việc render lại bình thường.

Over-optimization gây hại khi ta tối ưu quá sớm, trước khi có bằng chứng là app bị chậm. Code sẽ khó đọc hơn, nhiều hook lồng nhau, dependency array dài và dễ sai sót. Ví dụ quên thêm dependency có thể gây bug rất khó phát hiện. Ngoài ra, nếu lạm dụng React.memo, có thể làm logic update khó kiểm soát hơn.

Tóm lại, chỉ nên memoize khi đã xác định có vấn đề hiệu năng (dùng React DevTools Profiler để kiểm tra). Đừng tối ưu theo cảm tính. React vốn đã tối ưu khá tốt, nên ưu tiên code rõ ràng, dễ hiểu trước, sau đó mới tối ưu khi thật sự cần thiết.

Debounce và Throttle trong React dùng thế nào?

Debounce và Throttle là hai kỹ thuật dùng để kiểm soát việc một hàm bị gọi quá nhiều lần trong thời gian ngắn. Trong React, chúng thường được dùng khi xử lý các sự kiện như gõ input, scroll, resize hoặc search API.

Debounce nghĩa là chỉ thực hiện hàm sau khi người dùng dừng thao tác một khoảng thời gian nhất định. Ví dụ trong ô tìm kiếm, nếu người dùng gõ liên tục, ta không muốn mỗi ký tự lại gọi API một lần. Thay vào đó, ta chờ người dùng ngừng gõ 500ms rồi mới gọi API. Như vậy sẽ giảm số lần request và tiết kiệm tài nguyên.

Throttle nghĩa là giới hạn số lần gọi hàm trong một khoảng thời gian cố định. Ví dụ khi người dùng scroll trang, sự kiện scroll có thể chạy hàng chục lần mỗi giây. Nếu dùng throttle 1 giây, thì dù scroll liên tục, hàm cũng chỉ chạy tối đa 1 lần mỗi giây. Điều này hữu ích khi cần cập nhật vị trí, tính toán layout hoặc gửi tracking event.

Trong React, ta thường dùng debounce hoặc throttle kết hợp với useCallback để giữ function ổn định giữa các lần render. Có thể tự viết logic bằng setTimeout, hoặc dùng thư viện như lodash. Quan trọng là phải đảm bảo cleanup đúng cách (clearTimeout) nếu component unmount để tránh memory leak.

Tóm lại, debounce phù hợp khi muốn chờ người dùng “dừng lại” rồi mới xử lý (như search input). Throttle phù hợp khi muốn giới hạn tần suất xử lý trong các sự kiện xảy ra liên tục (như scroll, resize). Cả hai đều giúp cải thiện hiệu năng và tránh gọi hàm quá nhiều lần không cần thiết.

Virtualization là gì? Khi nào cần dùng (ví dụ: danh sách 10k items)?

Virtualization trong React là kỹ thuật chỉ render những phần tử đang hiển thị trên màn hình, thay vì render toàn bộ danh sách. Ví dụ nếu bạn có 10.000 items trong một list, React không nhất thiết phải tạo 10.000 DOM node cùng lúc. Thay vào đó, nó chỉ render khoảng 20–30 item đang nhìn thấy, và khi người dùng scroll thì thay thế các item cũ bằng item mới.

Vấn đề khi render 10.000 items cùng lúc là trình duyệt phải tạo rất nhiều DOM node, tốn bộ nhớ và thời gian render. Điều này có thể làm trang web bị lag, đặc biệt trên máy yếu. Virtualization giúp giảm số lượng DOM node thực sự tồn tại tại một thời điểm, từ đó cải thiện hiệu năng đáng kể.

Cách hoạt động cơ bản là tạo một “khung nhìn” (viewport) có chiều cao cố định. Bên trong đó, ta tính toán xem hiện tại người dùng đang scroll tới vị trí nào, và chỉ render các item tương ứng với vùng nhìn thấy đó. Các item phía trên và phía dưới sẽ được thay bằng khoảng trống có chiều cao tương ứng, để thanh scroll vẫn hoạt động bình thường như có đủ 10.000 items.

Trong React, người ta thường dùng các thư viện như react-window hoặc react-virtualized để xử lý virtualization thay vì tự viết từ đầu. Các thư viện này đã tối ưu sẵn và rất phổ biến trong các hệ thống có danh sách lớn như bảng dữ liệu, log hệ thống, chat history, hoặc dashboard có nhiều dòng dữ liệu.

Tóm lại, bạn nên dùng virtualization khi danh sách rất lớn (ví dụ vài nghìn đến hàng chục nghìn items) và có hiện tượng lag khi render. Nếu chỉ vài chục hoặc vài trăm item thì thường chưa cần thiết, vì thêm virtualization cũng làm code phức tạp hơn.

Làm sao tối ưu bundle size?

Tối ưu bundle size trong React nghĩa là giảm kích thước file JavaScript mà trình duyệt phải tải khi người dùng mở web. Bundle càng nhỏ thì thời gian load càng nhanh, đặc biệt quan trọng với mạng chậm hoặc thiết bị yếu.

Cách đầu tiên và quan trọng nhất là dùng code splitting. Thay vì gộp toàn bộ code vào một file lớn, ta chia nhỏ theo từng trang hoặc từng phần chức năng và chỉ load khi cần. Ví dụ, trang Admin chỉ được tải khi người dùng truy cập vào đó. Điều này giúp giảm kích thước file ban đầu.

Cách thứ hai là loại bỏ thư viện không cần thiết hoặc thay bằng thư viện nhẹ hơn. Ví dụ nếu chỉ dùng một vài hàm nhỏ từ một thư viện lớn, có thể import theo cách tree-shaking để chỉ lấy phần cần dùng. Hoặc cân nhắc xem có thật sự cần thư viện đó hay không, hay có thể tự viết một đoạn code nhỏ thay thế.

Thứ ba là tối ưu import. Không nên import toàn bộ một thư viện nếu chỉ dùng một phần nhỏ. Ví dụ thay vì import tất cả, chỉ import module cụ thể. Ngoài ra cần đảm bảo project build ở chế độ production để bật minify, loại bỏ code thừa và giảm kích thước file.

Thứ tư là tối ưu hình ảnh và asset. Ảnh quá lớn cũng làm bundle tăng kích thước. Có thể dùng lazy loading cho ảnh, nén ảnh trước khi đưa vào project hoặc dùng CDN.

Cuối cùng là kiểm tra bundle bằng các công cụ như bundle analyzer để xem phần nào đang chiếm nhiều dung lượng nhất. Từ đó mới quyết định tối ưu ở đâu. Tóm lại, tối ưu bundle size là kết hợp giữa chia nhỏ code, hạn chế thư viện nặng, import đúng cách và build production đúng chuẩn.

Tại sao inline function hoặc inline object trong props có thể gây re-render? Cách xử lý?

Khi bạn truyền inline function hoặc inline object vào props, ví dụ như `onClick={() => doSomething()}` hoặc `style={{ color: "red" }}`, thì mỗi lần component cha render, những function hoặc object đó sẽ được tạo mới lại từ đầu. Dù nội dung bên trong giống hệt, nhưng về mặt bộ nhớ (reference) thì nó là một object mới.

React so sánh props theo kiểu so sánh nông (shallow comparison). Nghĩa là nó chỉ so sánh reference chứ không so sánh sâu bên trong object. Vì vậy, nếu bạn truyền một object hoặc function mới mỗi lần render, React sẽ nghĩ rằng props đã thay đổi, và component con sẽ bị re-render, kể cả khi dữ liệu thực tế không đổi.

Vấn đề này thường rõ ràng khi component con được bọc bằng `React.memo`. `React.memo` chỉ ngăn re-render khi props không thay đổi reference. Nhưng nếu bạn truyền inline function hoặc object, reference luôn khác, nên `React.memo` không còn tác dụng.

Cách xử lý là dùng `useCallback` để ghi nhớ function và dùng `useMemo` để ghi nhớ object hoặc giá trị tính toán. Khi dependency không thay đổi, React sẽ giữ nguyên reference cũ, giúp component con không bị re-render không cần thiết. Ngoài ra, trong một số trường hợp đơn giản, bạn có thể định nghĩa function bên ngoài component hoặc tách nhỏ component để giảm ảnh hưởng.

Tóm lại, inline function và inline object tạo ra reference mới mỗi lần render, khiến React nghĩ rằng props thay đổi. Để tối ưu, có thể dùng `useCallback`, `useMemo` hoặc tổ chức lại component khi thật sự cần thiết.

Web Worker có thể được sử dụng trong React để tối ưu performance như thế nào?

Web Worker là một cơ chế của trình duyệt cho phép bạn chạy JavaScript ở một luồng (thread) riêng biệt, tách khỏi luồng chính (main thread). Trong React, main thread vừa phải xử lý render UI, vừa xử lý logic JavaScript. Nếu có một đoạn tính toán nặng chạy trên main thread, giao diện sẽ bị giật hoặc đứng hình.

Web Worker giúp giải quyết vấn đề này bằng cách chuyển những tác vụ nặng sang chạy ở background thread. Ví dụ như xử lý dữ liệu lớn, parse file CSV hàng chục nghìn dòng, mã hóa/giải mã dữ liệu, tính toán phức tạp hoặc xử lý hình ảnh. Khi đó, main thread vẫn rảnh để render UI và xử lý tương tác người dùng, nên trải nghiệm sẽ mượt hơn.

Trong React, bạn có thể tạo một file worker riêng. Khi cần xử lý nặng, component sẽ gửi dữ liệu sang worker thông qua cơ chế `postMessage`. Worker xử lý xong sẽ gửi kết quả ngược lại. Khi nhận kết quả, bạn cập nhật state bình thường. Như vậy, phần tính toán không làm block UI.

Điểm quan trọng là Web Worker không thể truy cập trực tiếp DOM hoặc state của React. Nó chỉ nhận dữ liệu đầu vào và trả về kết quả. Vì vậy, bạn nên dùng Web Worker cho các tác vụ thuần tính toán, không liên quan trực tiếp đến giao diện.

Tóm lại, Web Worker trong React được dùng để chuyển các tác vụ nặng ra khỏi main thread, giúp tránh block UI và cải thiện performance, đặc biệt khi xử lý dữ liệu lớn hoặc tính toán phức tạp.

Redux Core

Redux hoạt động theo nguyên lý gì?

Redux hoạt động theo nguyên lý quản lý state tập trung và một chiều (one-way data flow). Nghĩa là toàn bộ state của ứng dụng được lưu trong một nơi duy nhất gọi là store, thay vì mỗi component tự quản lý và truyền state lòng vòng qua nhiều cấp.

Nguyên lý cốt lõi của Redux gồm ba phần chính: store, action và reducer. Khi người dùng thực hiện một hành động (ví dụ click nút, submit form), ta sẽ dispatch một action. Action chỉ là một object mô tả “điều gì đã xảy ra”, ví dụ type: "ADD_TODO".

Sau khi dispatch, action sẽ được gửi tới reducer. Reducer là một hàm thuần (pure function), nhận vào state hiện tại và action, sau đó trả về state mới. Quan trọng là reducer không được thay đổi trực tiếp state cũ mà phải tạo ra state mới. Điều này giúp Redux dễ debug và theo dõi thay đổi.

Khi state trong store thay đổi, các component đang subscribe vào store sẽ được cập nhật lại (re-render). Như vậy, luồng dữ liệu của Redux luôn đi theo một vòng rõ ràng: User action → Dispatch action → Reducer xử lý → Tạo state mới → UI cập nhật.

Ưu điểm của nguyên lý này là dễ kiểm soát, dễ debug (vì mọi thay đổi đều thông qua action) và phù hợp với ứng dụng lớn có nhiều component cần dùng chung state. Tuy nhiên với ứng dụng nhỏ, Redux có thể hơi phức tạp và nhiều boilerplate.

Tóm lại, Redux hoạt động theo nguyên lý state tập trung, dữ liệu một chiều và mọi thay đổi đều phải đi qua action và reducer, giúp quản lý state một cách rõ ràng và có tổ chức.

Flow của Redux từ dispatch → reducer → store?

Flow của Redux có thể hiểu rất đơn giản là một vòng lặp một chiều, đi theo thứ tự: dispatch → reducer → store → UI cập nhật.

Đầu tiên là dispatch. Khi người dùng thực hiện một hành động, ví dụ click nút “Thêm sản phẩm”, component sẽ gọi dispatch và gửi lên một action. Action chỉ là một object mô tả sự kiện đã xảy ra, ví dụ có type là "ADD_PRODUCT" và kèm theo dữ liệu cần thêm.

Sau khi action được dispatch, Redux sẽ gửi action đó vào reducer. Reducer là một hàm nhận vào hai thứ: state hiện tại và action vừa được gửi lên. Dựa vào type của action, reducer sẽ quyết định phải xử lý thế nào và trả về một state mới. Quan trọng là reducer không sửa trực tiếp state cũ mà phải tạo ra bản sao mới.

Khi reducer trả về state mới, Redux sẽ cập nhật store bằng state đó. Store chính là nơi lưu toàn bộ state của ứng dụng. Khi store thay đổi, những component nào đang subscribe vào phần state đó sẽ tự động re-render để hiển thị dữ liệu mới.

Tóm lại, flow của Redux luôn theo một chiều rõ ràng: Component dispatch action → Reducer xử lý và tạo state mới → Store cập nhật → UI re-render. Nhờ luồng một chiều này, ta có thể dễ dàng theo dõi và debug vì mọi thay đổi state đều đi qua một quy trình cố định.

Tại sao reducer phải là pure function?

Reducer trong Redux phải là pure function vì Redux cần đảm bảo việc thay đổi state luôn rõ ràng, dự đoán được và dễ debug.

Pure function là hàm có hai đặc điểm chính. Thứ nhất, với cùng một input (state cũ và action), nó luôn trả về cùng một output (state mới). Thứ hai, nó không gây ra side effect, nghĩa là không được gọi API, không thay đổi biến bên ngoài, không sửa trực tiếp state cũ.

Nếu reducer không phải là pure function, ví dụ nó sửa trực tiếp state cũ hoặc có logic ngẫu nhiên như dùng Math.random(), thì kết quả mỗi lần chạy có thể khác nhau. Điều này làm cho ứng dụng khó dự đoán, khó debug và khó test. Redux được thiết kế để mọi thay đổi state đều minh bạch và có thể theo dõi, nên reducer bắt buộc phải thuần túy.

Ngoài ra, Redux dựa vào việc so sánh state cũ và state mới để biết có cần re-render hay không. Nếu reducer sửa trực tiếp state cũ (mutate), React có thể không phát hiện ra sự thay đổi, dẫn đến UI không cập nhật đúng. Việc luôn trả về một object mới giúp hệ thống hoạt động chính xác.

Tóm lại, reducer phải là pure function để đảm bảo tính ổn định, dễ test, dễ debug và giúp Redux hoạt động đúng theo nguyên lý dữ liệu một chiều.

Immutability trong Redux quan trọng thế nào?

Immutability trong Redux nghĩa là không được thay đổi trực tiếp state cũ, mà mỗi lần update phải tạo ra một bản sao mới của state. Đây là nguyên tắc rất quan trọng trong Redux vì nó ảnh hưởng trực tiếp đến cách hệ thống phát hiện thay đổi và cập nhật UI.

Redux (và React) thường kiểm tra sự thay đổi bằng cách so sánh reference giữa state cũ và state mới. Nếu bạn sửa trực tiếp object cũ (ví dụ thêm phần tử vào array cũ), thì reference không đổi. Khi đó Redux có thể nghĩ rằng state không thay đổi và UI sẽ không re-render đúng cách. Nhưng nếu bạn tạo một object hoặc array mới, reference sẽ khác và hệ thống sẽ biết rằng có thay đổi xảy ra.

Immutability cũng giúp cho việc debug dễ dàng hơn. Vì mỗi lần thay đổi đều tạo ra một state mới, ta có thể lưu lại lịch sử các state trước đó. Đây là lý do vì sao Redux có thể hỗ trợ tính năng time-travel debugging (quay lại các bước trước đó). Nếu state bị mutate, việc này gần như không thể thực hiện chính xác.

Ngoài ra, immutability làm cho reducer trở nên predictable hơn. Khi ta biết chắc rằng state cũ không bị sửa đổi, ta có thể yên tâm rằng mọi thay đổi đều rõ ràng và kiểm soát được.

Tóm lại, immutability trong Redux rất quan trọng vì nó giúp phát hiện thay đổi chính xác, đảm bảo UI cập nhật đúng, hỗ trợ debug tốt hơn và giữ cho luồng dữ liệu luôn rõ ràng, ổn định.

Sự khác nhau giữa Redux và Context API?

Redux và Context API đều dùng để chia sẻ state giữa nhiều component mà không cần truyền props qua nhiều tầng. Tuy nhiên, mục đích và cách sử dụng của hai cái này khác nhau khá nhiều.

Context API là tính năng có sẵn trong React, giúp truyền dữ liệu từ component cha xuống nhiều component con mà không cần props drilling. Nó phù hợp cho những dữ liệu dùng chung nhưng ít thay đổi, ví dụ theme (dark/light), ngôn ngữ (i18n), hoặc thông tin user hiện tại. Context đơn giản, dễ dùng và không cần cài thêm thư viện.

Redux là một thư viện quản lý state bên ngoài React. Nó có store trung tâm, action, reducer và luồng dữ liệu một chiều rất rõ ràng. Redux phù hợp với ứng dụng lớn, có nhiều state phức tạp, nhiều nơi cùng cập nhật state và cần debug kỹ. Redux còn có middleware để xử lý async (như gọi API), logging, và nhiều công cụ hỗ trợ mạnh mẽ.

Một điểm khác biệt quan trọng là hiệu năng và kiểm soát update. Khi giá trị trong Context thay đổi, tất cả component dùng context đó sẽ re-render. Với Redux (đặc biệt khi dùng selector đúng cách), chỉ những component sử dụng phần state bị thay đổi mới re-render, giúp tối ưu tốt hơn trong app lớn.

Tóm lại, Context API phù hợp cho state đơn giản, phạm vi nhỏ và ít thay đổi. Redux phù hợp cho ứng dụng lớn, state phức tạp, cần cấu trúc rõ ràng và khả năng debug mạnh. Trong nhiều dự án nhỏ và vừa, Context là đủ; còn khi hệ thống lớn và phức tạp hơn, Redux sẽ phát huy lợi thế.

Khi nào nên dùng Redux? Khi nào không cần?

Redux nên được dùng khi ứng dụng có state phức tạp và nhiều component ở nhiều nơi khác nhau cùng cần truy cập và cập nhật chung một dữ liệu. Ví dụ như hệ thống e-commerce có giỏ hàng, user login, danh sách sản phẩm, filter, pagination, nhiều màn hình cùng liên quan đến nhau. Khi đó, việc gom toàn bộ state về một store trung tâm giúp luồng dữ liệu rõ ràng, dễ kiểm soát và dễ debug.

Redux cũng phù hợp khi ứng dụng có nhiều logic xử lý phức tạp, nhiều action bất đồng bộ (gọi API nhiều bước), hoặc khi team lớn cần một cấu trúc chuẩn để mọi người làm việc thống nhất. Ngoài ra, nếu dự án yêu cầu logging, tracking thay đổi state, hoặc cần debug nâng cao, Redux sẽ rất hữu ích vì mọi thay đổi đều đi qua action và reducer.

Ngược lại, không nên dùng Redux khi ứng dụng nhỏ hoặc state đơn giản. Ví dụ một trang web chỉ có vài form, vài state local trong từng component, thì dùng useState, useReducer hoặc Context API là đủ. Nếu dùng Redux trong trường hợp này, bạn sẽ phải viết thêm nhiều boilerplate code không cần thiết, làm dự án phức tạp hơn.

Tóm lại, nên dùng Redux khi state toàn cục nhiều, phức tạp và cần quản lý chặt chẽ. Không cần dùng Redux khi ứng dụng nhỏ, ít chia sẻ state hoặc chỉ cần quản lý state cục bộ trong component. Quan trọng là chọn công cụ phù hợp với độ phức tạp của dự án, không nên dùng Redux chỉ vì “thói quen” hoặc “xu hướng”.

Single source of truth trong Redux mang lại lợi ích gì trong ứng dụng phức tạp?

Single source of truth trong Redux có nghĩa là toàn bộ state của ứng dụng được lưu tập trung trong một store duy nhất. Thay vì mỗi component tự giữ một phần state riêng rẽ khó kiểm soát, Redux gom tất cả dữ liệu quan trọng về một nơi để quản lý.

Trong ứng dụng phức tạp có nhiều component liên quan đến cùng một dữ liệu (ví dụ: thông tin user, giỏ hàng, quyền truy cập...), việc có một nguồn dữ liệu duy nhất giúp tránh

tình trạng lệch state. Nếu mỗi nơi tự giữ một bản copy dữ liệu, rất dễ xảy ra trường hợp một chỗ cập nhật nhưng chỗ khác không cập nhật theo. Single source of truth giúp đảm bảo mọi component đều đọc từ cùng một dữ liệu gốc.

Lợi ích lớn tiếp theo là dễ debug. Vì toàn bộ state nằm trong một store và thay đổi state chỉ thông qua action và reducer, bạn có thể theo dõi được chính xác khi nào state thay đổi và thay đổi như thế nào. Điều này đặc biệt hữu ích khi dùng các công cụ như Redux DevTools để xem lại lịch sử thay đổi (time-travel debugging).

Ngoài ra, kiến trúc này giúp code dễ bảo trì hơn khi ứng dụng lớn dần. Khi có bug hoặc cần thêm tính năng, bạn biết chắc mình cần chỉnh sửa ở đâu thay vì phải dò state nằm rải rác ở nhiều component. Logic cập nhật dữ liệu được tách rõ ràng khỏi UI, giúp hệ thống dễ mở rộng và dễ test hơn.

Tóm lại, single source of truth trong Redux giúp dữ liệu nhất quán, dễ debug, dễ mở rộng và đặc biệt phù hợp với các ứng dụng phức tạp có nhiều component dùng chung state.

Vì sao không nên lưu state có thể tính toán được (derived state) trực tiếp trong Redux store?

Derived state là những dữ liệu có thể tính toán lại từ state gốc. Ví dụ như danh sách sản phẩm đã lọc, tổng tiền giỏ hàng, số lượng item đang được chọn... Những giá trị này thực chất có thể được tính lại từ dữ liệu ban đầu thay vì phải lưu riêng.

Không nên lưu derived state trực tiếp trong Redux store vì dễ gây mất đồng bộ dữ liệu. Nếu state gốc thay đổi nhưng bạn quên cập nhật lại derived state tương ứng, ứng dụng sẽ hiển thị sai dữ liệu. Khi có nhiều nơi cùng phụ thuộc vào derived state, rủi ro này càng lớn.

Ngoài ra, việc lưu cả state gốc và state tính toán sẽ làm store phình to không cần thiết. Redux nên giữ dữ liệu “nguồn gốc” (raw data) càng đơn giản càng tốt. Càng ít dữ liệu trùng lặp, càng dễ quản lý và debug.

Một lý do quan trọng khác là tính toán derived state có thể thực hiện tại thời điểm cần dùng, thông qua selector. Thư viện như Reselect cho phép memoize kết quả tính toán, nghĩa là chỉ tính lại khi dữ liệu đầu vào thay đổi. Như vậy vừa đảm bảo hiệu năng, vừa tránh phải lưu trữ dư thừa trong store.

Tóm lại, Redux store nên giữ dữ liệu gốc làm “single source of truth”. Những giá trị có thể tính toán lại thì nên tính khi cần, thay vì lưu trực tiếp, để tránh sai lệch dữ liệu và giúp hệ thống gọn gàng, dễ bảo trì hơn.

Redux nâng cao

Redux Thunk vs Redux Saga khác nhau thế nào?

Redux Thunk và Redux Saga đều là middleware trong Redux, dùng để xử lý logic bất đồng bộ như gọi API, delay, hoặc xử lý nhiều bước trước khi cập nhật state. Vì reducer phải là pure function nên không được gọi API trực tiếp, nên ta cần middleware như Thunk hoặc Saga để xử lý phần này.

Redux Thunk hoạt động đơn giản hơn. Thay vì dispatch một object action bình thường, ta có thể dispatch một function. Function này sẽ nhận dispatch và getState làm tham số, từ đó có thể gọi API rồi sau khi có kết quả mới dispatch action thật sự vào reducer. Thunk dễ học, dễ dùng và phù hợp với các ứng dụng vừa và nhỏ. Tuy nhiên khi logic async phức tạp (nhiều bước, nhiều nhánh, phụ thuộc lẫn nhau), code Thunk có thể trở nên khó đọc.

Redux Saga phức tạp hơn nhưng mạnh mẽ hơn. Saga sử dụng generator function (function có dấu *) để mô tả các bước bất đồng bộ theo kiểu “giống như synchronous”. Thay vì gọi API trực tiếp, ta dùng các effect như call, put, takeLatest... để điều khiển luồng xử lý. Saga rất phù hợp khi có nhiều logic phức tạp như hủy request cũ, retry, chạy song song nhiều tác vụ, hoặc kiểm soát chặt chẽ thứ tự thực thi.

Nói đơn giản, Thunk giống như viết async/await bình thường trong Redux, còn Saga giống như xây một hệ thống điều phối riêng cho các tác vụ bất đồng bộ. Thunk nhẹ và dễ dùng hơn, Saga mạnh và linh hoạt hơn nhưng khó học hơn.

Tóm lại, nếu dự án không quá phức tạp thì Redux Thunk là đủ và dễ bảo trì. Nếu ứng dụng lớn, có nhiều luồng async phức tạp và cần kiểm soát nâng cao, Redux Saga sẽ phù hợp hơn.

Middleware trong Redux hoạt động ra sao?

Middleware trong Redux là một lớp trung gian nằm giữa lúc bạn dispatch action và lúc action đó đi vào reducer. Nói đơn giản, nó giống như một “người kiểm tra” đứng giữa đường đi của action để xử lý thêm trước khi reducer nhận được.

Bình thường flow của Redux là: dispatch → reducer → store cập nhật. Nhưng khi có middleware, flow sẽ thành: dispatch → middleware → reducer → store. Middleware có thể chặn, sửa, log hoặc xử lý thêm logic trước khi action tới reducer.

Ví dụ phổ biến nhất là xử lý bất đồng bộ như gọi API. Reducer phải là pure function nên không được gọi API. Vì vậy, ta dùng middleware (như Redux Thunk hoặc Redux Saga) để

xử lý phần async này. Middleware sẽ nhận action, gọi API nếu cần, rồi sau đó dispatch một action khác khi có kết quả.

Ngoài async, middleware còn có thể dùng để logging (ghi log mỗi lần dispatch), bắt lỗi, hoặc thêm thông tin vào action. Điểm quan trọng là middleware không thay thế reducer, mà chỉ mở rộng khả năng của Redux mà vẫn giữ nguyên nguyên lý một chiều.

Tóm lại, middleware là lớp trung gian giúp Redux xử lý thêm logic (đặc biệt là async) trước khi state được cập nhật. Nó giúp hệ thống linh hoạt hơn mà vẫn giữ được cấu trúc rõ ràng và dễ kiểm soát.

Cách xử lý API call với Redux Toolkit?

Khi dùng Redux Toolkit (RTK), việc xử lý API call trở nên đơn giản hơn rất nhiều so với Redux truyền thống, vì RTK đã giảm bớt rất nhiều boilerplate code.

Cách phổ biến nhất là dùng `createAsyncThunk`. Đây là một hàm giúp bạn tạo action bất đồng bộ. Bên trong `createAsyncThunk`, bạn viết logic gọi API (ví dụ `fetch` hoặc `axios`). Khi gọi API, Redux Toolkit sẽ tự động tạo ra ba trạng thái: `pending` (đang gọi), `fulfilled` (thành công) và `rejected` (thất bại). Bạn chỉ cần xử lý ba trạng thái này trong `extraReducers` của `slice` để cập nhật state tương ứng như `loading`, `data` hoặc `error`.

Flow sẽ như sau: component dispatch async thunk → thunk gọi API → RTK tự động dispatch action `pending` → khi API xong sẽ dispatch `fulfilled` hoặc `rejected` → reducer cập nhật state → UI re-render. Nhờ vậy bạn không phải tự viết quá nhiều action type như trước đây.

Ngoài `createAsyncThunk`, Redux Toolkit còn có RTK Query. Đây là một giải pháp mạnh hơn, gần như chuyên để quản lý API call. RTK Query tự động handle caching, loading, error, refetch và nhiều thứ khác. Bạn chỉ cần định nghĩa endpoint, còn việc gọi API và quản lý state gần như tự động.

Tóm lại, với Redux Toolkit, cách cơ bản để xử lý API là dùng `createAsyncThunk` trong `slice`. Nếu dự án có nhiều API và cần quản lý nâng cao như caching và auto refetch, thì RTK Query là lựa chọn rất tốt và hiện đại.

`createSlice` trong Redux Toolkit giúp gì?

`createSlice` trong Redux Toolkit giúp chúng ta gom toàn bộ logic liên quan đến một phần state vào một chỗ duy nhất, thay vì phải tách riêng action type, action creator và reducer

như Redux truyền thống. Nói đơn giản, nó giúp viết Redux ngắn gọn và dễ đọc hơn rất nhiều.

Trước đây khi dùng Redux thuần, mỗi lần tạo một feature mới, ta phải khai báo action type dạng string, viết action creator, rồi viết reducer với switch-case khá dài. Với createSlice, ta chỉ cần định nghĩa tên slice, state ban đầu (initialState) và các hàm reducer bên trong một object. Redux Toolkit sẽ tự động tạo action creator và action type tương ứng cho ta.

Một điểm mạnh nữa là createSlice cho phép “viết như thể đang mutate state”, nhưng thực tế bên trong nó dùng thư viện Immer để đảm bảo immutability. Điều này giúp code ngắn hơn và dễ hiểu hơn, vì ta không phải tự spread object hoặc array quá nhiều lần.

Ngoài ra, createSlice còn dễ kết hợp với createAsyncThunk để xử lý API call thông qua phân extraReducers. Nhờ vậy, toàn bộ logic liên quan đến một feature (state, sync action, async action) đều nằm chung trong một file, rất rõ ràng và dễ bảo trì.

Tóm lại, createSlice giúp giảm boilerplate, tổ chức code gọn gàng theo từng feature và giúp viết reducer dễ dàng hơn mà vẫn đảm bảo nguyên tắc immutability của Redux.

RTK Query là gì? Khi nào nên dùng thay vì Thunk?

RTK Query là một công cụ được tích hợp sẵn trong Redux Toolkit, chuyên dùng để xử lý API call và quản lý dữ liệu từ server. Có thể hiểu nó là một lớp “tự động hóa” cho việc gọi API, lưu cache, xử lý loading và error, thay vì ta phải tự viết nhiều logic như khi dùng Thunk.

Khi dùng Redux Thunk, bạn phải tự viết createAsyncThunk, tự quản lý state như loading, success, error trong slice. Nếu có nhiều API, bạn sẽ phải lặp lại khá nhiều cấu trúc giống nhau. RTK Query giúp đơn giản hóa việc này. Bạn chỉ cần định nghĩa các endpoint (ví dụ: getUsers, createUser...), còn lại nó sẽ tự động tạo hook như useGetUsersQuery để dùng trực tiếp trong component.

Một điểm mạnh của RTK Query là nó có cơ chế caching thông minh. Nếu bạn gọi cùng một API nhiều lần với cùng tham số, nó có thể dùng lại dữ liệu cũ thay vì gọi lại server. Nó cũng hỗ trợ auto refetch khi cần, invalidation cache khi update dữ liệu và nhiều tính năng tối ưu sẵn có. Những thứ này nếu dùng Thunk bạn phải tự xử lý.

Bạn nên dùng RTK Query khi ứng dụng có nhiều API, cần quản lý dữ liệu server phức tạp, cần caching và đồng bộ dữ liệu tự động. Ngược lại, nếu logic async đơn giản hoặc bạn cần

kiểm soát luồng xử lý đặc biệt (ví dụ nhiều bước xử lý custom trước và sau API), thì Thunk có thể linh hoạt hơn.

Tóm lại, RTK Query phù hợp cho việc quản lý dữ liệu từ server theo kiểu chuẩn CRUD và muốn tối ưu sẵn về caching, loading, refetch. Thunk phù hợp khi bạn cần tự kiểm soát logic bất đồng bộ một cách linh hoạt và chi tiết hơn.

Làm sao normalize dữ liệu trong Redux store?

Normalize dữ liệu trong Redux store nghĩa là tổ chức dữ liệu theo dạng “phẳng” (flat), tránh lồng object quá sâu. Mục tiêu là giúp việc update, truy xuất và quản lý state dễ hơn, đặc biệt khi dữ liệu có quan hệ với nhau như user, post, comment.

Ví dụ, thay vì lưu một danh sách bài viết mà trong mỗi bài viết lại chứa đầy đủ thông tin user và comment bên trong (nested sâu nhiều tầng), ta sẽ tách riêng ra thành nhiều “bảng” giống như trong database. Ta có một object lưu tất cả user theo id, một object lưu tất cả post theo id, và mỗi post chỉ lưu userId hoặc danh sách commentId. Cách này giúp tránh trùng lặp dữ liệu.

Lợi ích của normalize là khi cần cập nhật một user, ta chỉ cần sửa một chỗ trong store, thay vì phải đi tìm và sửa ở nhiều nơi lồng sâu. Điều này cũng giúp reducer đơn giản hơn và giữ đúng nguyên tắc immutability dễ hơn.

Trong Redux Toolkit, ta có thể tự normalize bằng cách lưu dữ liệu theo dạng: entities: { byId: {}, allIds: [] }. Hoặc dùng công cụ hỗ trợ như createEntityAdapter của Redux Toolkit. Công cụ này giúp quản lý danh sách theo id, tự tạo sẵn các hàm như addOne, updateOne, removeOne và giữ dữ liệu ở dạng chuẩn hóa.

Tóm lại, normalize dữ liệu là cách tổ chức store giống như database: tách riêng từng loại entity, lưu theo id và liên kết bằng id thay vì lồng object sâu. Điều này giúp store gọn gàng, dễ update và dễ bảo trì hơn trong ứng dụng lớn.

Redux middleware chain hoạt động theo thứ tự như thế nào? Nếu một middleware không gọi next(action) thì chuyện gì xảy ra?

Redux middleware hoạt động theo cơ chế “chuỗi” (chain). Khi bạn dispatch một action, action đó sẽ không đi thẳng vào reducer ngay, mà sẽ đi qua từng middleware theo đúng thứ tự mà bạn cấu hình trong store. Middleware đầu tiên nhận action trước, sau đó nếu nó gọi next(action) thì action mới được chuyển sang middleware tiếp theo. Cứ như vậy cho đến middleware cuối cùng, rồi cuối cùng action mới tới reducer để cập nhật state.

Bạn có thể hình dung middleware giống như một dây chuyền xử lý. Mỗi middleware có cơ hội kiểm tra, chỉnh sửa, log, hoặc thậm chí chặn action. Ví dụ: một middleware có thể dùng để log action, một middleware khác xử lý async như gọi API, rồi sau đó mới cho action đi tiếp.

Nếu một middleware không gọi `next(action)`, thì chuỗi sẽ bị dừng lại tại đó. Action sẽ không đi tiếp tới các middleware phía sau và cũng không tới reducer. Điều này có nghĩa là state sẽ không được cập nhật (trừ khi middleware đó tự dispatch một action khác). Đây là cách middleware có thể “chặn” một action, ví dụ khi phát hiện user chưa đăng nhập thì không cho phép thực hiện action nhạy cảm.

Tóm lại, middleware chạy theo thứ tự khai báo, mỗi middleware phải gọi `next(action)` để action tiếp tục đi xuống chuỗi. Nếu không gọi, action sẽ bị chặn lại và reducer sẽ không nhận được action đó.

Sự khác biệt giữa optimistic update và pessimistic update trong Redux là gì? Khi nào nên dùng mỗi loại?

Optimistic update là cách cập nhật UI trước khi server trả về kết quả thành công. Nghĩa là khi người dùng thực hiện một hành động, ví dụ nhấn “Like” hoặc “Thêm vào giỏ hàng”, Redux sẽ cập nhật state ngay lập tức để giao diện thay đổi tức thì, rồi sau đó mới gửi request lên server. Nếu server báo lỗi, ta sẽ rollback lại state cũ. Cách này giúp ứng dụng cảm giác rất nhanh và mượt vì người dùng không phải chờ phản hồi từ backend.

Ngược lại, pessimistic update là cách chờ server xử lý xong rồi mới cập nhật state trong Redux. Khi người dùng thực hiện hành động, ta thường hiển thị loading hoặc disable nút bấm. Chỉ khi server trả về thành công thì mới dispatch action để cập nhật state. Cách này an toàn hơn vì state trên client luôn đồng bộ với server, nhưng trải nghiệm có thể chậm hơn do phải chờ phản hồi.

Optimistic update phù hợp khi xác suất thất bại thấp và việc rollback không quá phức tạp, ví dụ như like bài viết, đánh dấu đã đọc, thêm vào danh sách tạm thời. Nó cải thiện UX rất tốt trong các thao tác nhỏ, thường xuyên. Ngược lại, pessimistic update nên dùng khi thao tác có rủi ro cao hoặc liên quan đến dữ liệu quan trọng như thanh toán, chuyển tiền, cập nhật dữ liệu nhạy cảm. Trong những trường hợp này, ưu tiên tính chính xác và an toàn hơn là tốc độ hiển thị.

Tóm lại, optimistic update ưu tiên trải nghiệm nhanh và mượt, chấp nhận xử lý rollback nếu có lỗi. Pessimistic update ưu tiên sự chắc chắn và đồng bộ với server, chấp nhận chờ phản hồi trước khi cập nhật UI.

Thực tế dự án

Bạn sẽ xử lý authentication flow trong React + Redux như thế nào? (Gợi ý: lưu token ở đâu? refresh token? intercept axios?)

Khi xử lý authentication flow trong React + Redux, mình thường chia ra thành các bước rõ ràng: login, lưu token, gọi API kèm token, xử lý hết hạn token và logout.

Đầu tiên là login. Khi người dùng nhập email và password, component sẽ dispatch một async action (ví dụ dùng createAsyncThunk) để gọi API login. Nếu thành công, server thường trả về access token (và có thể có refresh token). Sau đó mình sẽ lưu thông tin user và access token vào Redux store để các component khác có thể biết trạng thái đã đăng nhập.

Về việc lưu token ở đâu, có hai cách phổ biến. Access token có thể lưu trong memory (Redux store) để giảm rủi ro bảo mật. Nếu cần giữ trạng thái đăng nhập khi reload trang, có thể lưu thêm vào localStorage. Tuy nhiên localStorage dễ bị tấn công XSS, nên trong hệ thống bảo mật cao, refresh token thường được lưu trong httpOnly cookie do server set, để JavaScript không truy cập được.

Khi gọi API, mình thường dùng axios và tạo một instance riêng. Trong axios instance này, mình cấu hình interceptor. Interceptor request sẽ tự động gắn access token vào header Authorization cho mọi request. Như vậy không cần gắn thủ công ở từng chỗ.

Về refresh token, khi server trả về lỗi 401 (token hết hạn), interceptor response sẽ bắt lỗi này. Lúc đó, nó sẽ tự động gọi API refresh token để lấy access token mới. Nếu refresh thành công, nó cập nhật token mới vào store rồi retry lại request ban đầu. Nếu refresh thất bại (ví dụ refresh token cũng hết hạn), thì dispatch logout và chuyển người dùng về trang login.

Ngoài ra, khi app khởi động lại (ví dụ reload trang), mình sẽ kiểm tra xem có token trong localStorage hoặc cookie không. Nếu có, có thể gọi API “get current user” để xác thực lại và cập nhật Redux store.

Tóm lại, authentication flow thường gồm: login → lưu token → dùng axios interceptor để tự động gắn token → xử lý 401 để refresh token → nếu thất bại thì logout. Quan trọng nhất là cân bằng giữa tiện lợi và bảo mật khi quyết định lưu token ở đâu.

Nếu một ứng dụng React lớn bị chậm, bạn sẽ debug và tối ưu từ đâu?

Nếu một ứng dụng React lớn bị chậm, việc đầu tiên mình làm không phải là tối ưu ngay, mà là xác định nguyên nhân chính xác. Tối ưu “mù” rất dễ làm code phức tạp hơn mà không cải thiện được nhiều.

Bước đầu tiên là dùng React DevTools, đặc biệt là tab Profiler. Mình sẽ record lại một thao tác (ví dụ chuyển trang, click button, nhập input) rồi xem component nào render nhiều lần, component nào tốn nhiều thời gian render nhất. Điều này giúp xác định có re-render không cần thiết hay không.

Nếu thấy nhiều component re-render liên tục, mình sẽ kiểm tra props có bị thay đổi reference không (object, array, function tạo mới mỗi lần render). Khi cần, mình mới dùng React.memo, useMemo hoặc useCallback để giảm re-render. Nhưng chỉ làm khi thật sự thấy nó gây ảnh hưởng.

Tiếp theo, mình sẽ kiểm tra danh sách lớn (ví dụ table có hàng nghìn dòng). Nếu render quá nhiều item cùng lúc, có thể cần dùng virtualization để chỉ render phần đang hiển thị. Đây là nguyên nhân phổ biến gây lag UI.

Ngoài phần render, mình cũng kiểm tra network. Mở tab Network trong DevTools để xem có API nào gọi quá nhiều lần không, có request bị lặp hay không. Nếu có, cần kiểm tra lại dependency của useEffect hoặc logic dispatch action.

Ngoài ra, mình sẽ kiểm tra bundle size. Nếu file JavaScript quá lớn, thời gian load ban đầu sẽ chậm. Khi đó có thể cần code splitting, lazy loading hoặc loại bỏ thư viện nặng không cần thiết.

Cuối cùng, nếu ứng dụng chậm do xử lý logic nặng (ví dụ tính toán lớn), có thể cân nhắc tối ưu thuật toán hoặc chuyển xử lý sang Web Worker để không block UI.

Tóm lại, quy trình sẽ là: dùng Profiler để tìm điểm nghẽn → kiểm tra re-render → kiểm tra danh sách lớn → kiểm tra API call → kiểm tra bundle size → tối ưu đúng chỗ. Luôn đo lường trước rồi mới tối ưu, tránh làm phức tạp hệ thống một cách không cần thiết.

Làm sao bạn thiết kế cơ chế phân quyền (role-based access control) trong React app?

Khi thiết kế cơ chế phân quyền theo role (RBAC) trong React app, ý tưởng cốt lõi là: backend quyết định user có quyền gì, còn frontend chỉ dựa vào thông tin đó để hiển thị hoặc ẩn các chức năng tương ứng. Frontend không nên tự “tin” rằng mình có thể bảo mật hoàn toàn, vì kiểm soát thật sự phải nằm ở server.

Bước đầu tiên là sau khi user đăng nhập, backend sẽ trả về thông tin như role (ví dụ: admin, manager, user) hoặc danh sách permissions (ví dụ: can_view_user, can_edit_user). Thông tin này thường được lưu vào Redux store hoặc context để toàn bộ ứng dụng có thể truy cập. Từ đó, các component có thể đọc role/permission và quyết định có render nội dung hay không.

Ở cấp độ UI, bạn có thể tạo một component wrapper, ví dụ như ProtectedRoute hoặc RequirePermission. Component này sẽ kiểm tra role trước khi cho phép truy cập vào một trang. Nếu không đủ quyền thì redirect sang trang “Unauthorized” hoặc trang login. Với các nút bấm nhỏ trong UI, bạn cũng có thể kiểm tra permission trước khi render, ví dụ chỉ hiển thị nút “Delete” nếu user có quyền xóa.

Để tránh rải logic kiểm tra quyền khắp nơi, bạn nên tách phần kiểm tra quyền thành một utility function như hasPermission(user, action) hoặc một custom hook như usePermission. Điều này giúp code dễ bảo trì và dễ mở rộng khi số lượng role tăng lên.

Tuy nhiên, cần nhớ rằng frontend chỉ nên dùng để ẩn/hiển thị và cải thiện UX. Backend vẫn phải kiểm tra quyền lại ở mỗi API. Nếu không, người dùng có thể gọi API trực tiếp mà bỏ qua UI.

Tóm lại, thiết kế RBAC trong React thường gồm: lưu role/permission sau login, dùng wrapper để bảo vệ route, kiểm tra permission khi render UI, và luôn đảm bảo backend là nơi kiểm soát quyền thực sự.

Nếu cần migrate từ Redux sang Redux Toolkit hoặc sang một state management khác (ví dụ: Zustand), bạn sẽ tiếp cận như thế nào để giảm rủi ro?

Khi migrate từ Redux sang Redux Toolkit (RTK) hoặc sang một state management khác như Zustand, nguyên tắc quan trọng nhất là không nên “đập đi xây lại” một lần. Với ứng dụng lớn, cách an toàn nhất là migrate từng phần nhỏ, giữ cho hệ thống vẫn chạy ổn định trong suốt quá trình chuyển đổi.

Nếu chuyển từ Redux thuần sang Redux Toolkit, rủi ro thường thấp hơn vì RTK thực chất là cách viết Redux hiện đại và gọn hơn. Lúc này, bạn có thể giữ nguyên store cũ, rồi dần dần chuyển từng reducer sang dạng createSlice. Bạn cũng có thể migrate async logic từ redux-thunk hoặc saga sang createAsyncThunk từng phần. Mỗi lần chuyển một slice, test kỹ rồi mới chuyển tiếp slice khác. Vì cùng nền tảng Redux nên có thể chạy song song reducer cũ và mới trong cùng store.

Nếu chuyển sang một state management khác như Zustand, mức độ thay đổi lớn hơn vì kiến trúc khác. Lúc này nên bắt đầu bằng cách phân tích state hiện tại: tách rõ global state nào thật sự cần, state nào có thể đưa về local component. Sau đó chọn một module ít phụ thuộc để thử migrate trước, ví dụ một feature độc lập. Giữ Redux cho phần còn lại, và dùng Zustand cho feature mới. Khi đã ổn định và team quen cách viết mới, mới tiếp tục migrate các phần khác.

Trong cả hai trường hợp, việc viết test trước khi migrate là rất quan trọng. Nếu có unit test cho reducer, selector, hoặc integration test cho flow chính, bạn sẽ tự tin hơn khi refactor. Ngoài ra, cần đảm bảo API giữa UI và state layer không bị thay đổi quá nhiều, ví dụ giữ nguyên shape dữ liệu trả về để tránh phải sửa hàng loạt component.

Cuối cùng, nên lên kế hoạch rõ ràng: xác định phạm vi migrate, thứ tự ưu tiên, và thời gian thực hiện. Không nên vừa thêm feature lớn vừa migrate state management cùng lúc vì sẽ khó kiểm soát lỗi.

Tóm lại, để giảm rủi ro khi migrate, bạn nên làm từng bước nhỏ, có test bảo vệ, có thể chạy song song hai hệ thống trong giai đoạn chuyển tiếp, và ưu tiên migrate những phần ít phụ thuộc trước.

Tổng hợp câu hỏi:

1. React Core

Virtual DOM là gì? Cơ chế diffing của React hoạt động như thế nào?

Tại sao key lại quan trọng trong list rendering? Nếu dùng index làm key thì rủi ro gì?

Controlled vs Uncontrolled component? Khi nào nên dùng mỗi loại?

Synthetic Event trong React là gì? Nó khác gì native DOM event?

React lifecycle (Class component) vs Hooks lifecycle mapping?

useEffect hoạt động như thế nào? Dependency array ảnh hưởng ra sao?

Cleanup function trong useEffect dùng để làm gì? Ví dụ thực tế?

useMemo vs useCallback khác nhau thế nào? Khi nào KHÔNG nên dùng?

React.memo hoạt động ra sao? So sánh với useMemo?

Lifting state up là gì? Khi nào nên dùng Context thay vì props drilling?

Sự khác nhau giữa useRef và useState là gì? Khi nào nên dùng useRef thay vì useState để tránh re-render?

Batch update trong React là gì? React xử lý batching state update khác nhau thế nào giữa event handler và async code?

2. React Advanced

React concurrent rendering là gì? (React 18)

StrictMode trong React dùng để làm gì? Tại sao useEffect chạy 2 lần ở dev?

Suspense hoạt động như thế nào? Có thể dùng cho data fetching không?

Code splitting trong React làm như thế nào?

Lazy loading component hoạt động ra sao?

Error Boundary là gì? Tại sao Hook không bắt được error như Class component?
Portals là gì? Dùng trong trường hợp nào?
Higher Order Component (HOC) vs Render Props vs Hooks?
Cách thiết kế reusable component library trong dự án lớn?
Cách tổ chức folder structure cho dự án React lớn?
useTransition trong React 18 dùng để làm gì? Nó giúp cải thiện UX trong trường hợp nào?
Server Components trong React là gì? Khác gì so với Client Components?

3. Performance Optimization

Làm sao để detect re-render không cần thiết?
Tại sao component bị re-render dù props không thay đổi?
Khi nào nên memoize? Khi nào over-optimization gây hại?
Debounce và Throttle trong React dùng thế nào?
Virtualization là gì? Khi nào cần dùng (ví dụ: danh sách 10k items)?
Làm sao tối ưu bundle size?
Tại sao inline function hoặc inline object trong props có thể gây re-render? Cách xử lý?
Web Worker có thể được sử dụng trong React để tối ưu performance như thế nào?

4. Redux Core

Redux hoạt động theo nguyên lý gì?
Flow của Redux từ dispatch → reducer → store?
Tại sao reducer phải là pure function?
Immutability trong Redux quan trọng thế nào?
Sự khác nhau giữa Redux và Context API?
Khi nào nên dùng Redux? Khi nào không cần?
Single source of truth trong Redux mang lại lợi ích gì trong ứng dụng phức tạp?
Vì sao không nên lưu state có thể tính toán được (derived state) trực tiếp trong Redux store?

5. Redux nâng cao

Redux Thunk vs Redux Saga khác nhau thế nào?
Middleware trong Redux hoạt động ra sao?
Cách xử lý API call với Redux Toolkit?
createSlice trong Redux Toolkit giúp gì?
RTK Query là gì? Khi nào nên dùng thay vì Thunk?
Làm sao normalize dữ liệu trong Redux store?
Redux middleware chain hoạt động theo thứ tự như thế nào? Nếu một middleware không gọi next(action) thì chuyện gì xảy ra?
Sự khác biệt giữa optimistic update và pessimistic update trong Redux là gì? Khi nào nên dùng mỗi loại?

6. Thực tế dự án

Bạn sẽ xử lý authentication flow trong React + Redux như thế nào? (Gợi ý: lưu token ở đâu? refresh token? intercept axios?)
Nếu một ứng dụng React lớn bị chậm, bạn sẽ debug và tối ưu từ đâu?
Làm sao bạn thiết kế cơ chế phân quyền (role-based access control) trong React app?
Nếu cần migrate từ Redux sang Redux Toolkit hoặc sang một state management khác (ví dụ: Zustand), bạn sẽ tiếp cận như thế nào để giảm rủi ro?

Bây giờ tôi sẽ đưa bạn từng câu hỏi, bạn hãy giúp tôi trả lời bằng 1 cách dễ hiểu nhất vì vốn tôi cũng không có quá rành React. Viết câu trả lời bằng các đoạn văn bản nhé, nhớ là đoạn.

40. Nếu một ứng dụng React lớn bị chậm, bạn sẽ debug và tối ưu từ đâu?